

客户端 / 服务器软件体系结构案例研究：银行系统

本章将介绍如何应用 COMET/UML 软件建模与设计方法来设计一个客户端 / 服务器软件体系结构（见第 15 章）：银行系统。此外，ATM 客户端的设计也被作为设计并发软件的一个例子（见第 18 章），而银行服务的设计则被作为顺序性面向对象软件设计的一个例子（见第 14 章）。

21.1 节介绍本案例中的问题。21.2 节给出“银行系统”的用例模型。21.3 节描述静态模型，覆盖了系统上下文以及实体类的静态建模。21.4 节介绍将系统组织为一组对象的方法。21.5 节介绍动态建模，其中为每个用例开发了交互图。21.6 节描述 ATM 的状态图。21.7 节到 21.14 节则描述“银行系统”的设计模型。

21.1 问题描述

一家银行拥有一些 ATM 机（自动取款机），这些 ATM 机分布在不同的地理位置并且通过广域网连接到一个中央服务器上。每一个 ATM 机由一个读卡器、一个吐钞器、一个键盘 / 显示器和一个凭条打印机组成。通过使用 ATM 机，客户能够从支票账户或储蓄账户提取现金、查询账户余额，或者在账户间转账。客户将一个 ATM 卡插入读卡器后会启动一个交易。ATM 卡背面的磁条里编码保存了该卡的卡号、生效期和失效期。如果一张 ATM 卡能够被系统识别，那么系统会验证这张卡以确定该卡没有过期、客户输入的 PIN 码（个人识别码）与系统中保留的 PIN 码匹配以及这张卡没有被挂失。客户可以尝试输入三次 PIN 码；如果第三次输入仍然错误，该卡会被没收。同时，被确认为挂失的卡也会被没收。

如果输入的 PIN 码通过了验证，那么该客户可以进行取款、查询或转账交易。在取款交易被许可之前，系统需确认被取款账户拥有足够的金额、取款额度未超过单日取款上限以及本地提款机中拥有足够的现金。如果该交易获得了许可，那么 ATM 机将提取指定的取款金额、打印包含交易信息的凭条并弹出 ATM 卡。在转账交易被许可前，系统需确认客户拥有至少两个账户以及待转出的账户中拥有足够的余额。对于被允许的查询和转账请求，ATM 机会打印凭条并弹出 ATM 卡。客户可以在任何时候取消交易，如果交易被取消，那么 ATM 卡也会被弹出。服务器中保留了所有的客户记录、账户记录以及借记卡记录。

一个 ATM 操作员可以开启或关闭 ATM 机，从而为 ATM 机补充现金或进行常规维护工作。这里假设开设账户、关闭账户和创建、更新和删除客户及借记卡记录的功能由现有的系统提供，这些并不是本案例中所涉及问题的一部分。

21.2 用例模型

用例在用例模型中描述。“银行系统”的用例模型中有两个参与者（actor），即“ATM 客户”（ATM Customer）和“操作员”（Operator），二者都是系统的用户。客户可以从支票账户或储蓄账户中取款、查询账户余额以及在账户间转账。

客户通过 ATM 读卡器和键盘与系统交互。参与者是客户，而非读卡器和键盘。这些输入设备使得客户可以启动用例并对系统的提示进行响应。凭条打印机和吐钞器是输出设备，它们也不是参与者，因为从这些用例中受益的是客户。

ATM 操作员可以关闭 ATM 机、为 ATM 的吐钞器补充现金并启动 ATM 机。参与者代表了由用户扮演的角色，因此系统中可以有多个客户和操作员。

现在考虑 ATM 操作员用例。一种方案是只有一个操作员用例，其中操作员关闭 ATM 机、添加现金然后启动 ATM 机。然而在现实中也有可能出现以下情况：操作员关闭 ATM 机是因为硬件出了问题而不是要添加现金；操作员启动 ATM 机是因为机器意外关闭。因此，与单个操作员用例的方案相比，一种更好的方案是将操作员用例分为三个用例，即“添加现金”（Add Cash）、“启动”（Startup）和“关闭”（Shutdown），如图 21-1 所示。

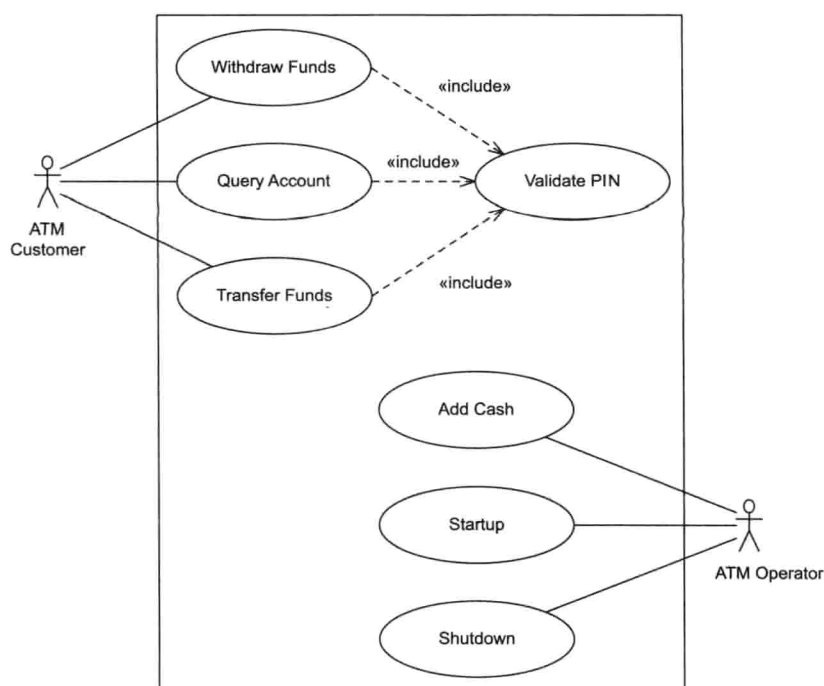


图 21-1 “银行系统”用例模型

再考虑由“ATM 客户”所触发的用例。一种可能的方案是为客户与系统之间所有的交互建立一个用例。然而，由客户发起的三种交易类型（即取款、查询和转账）相互之间非常独立，区分很明显。

因此，我们开始考虑三个独立的“ATM 客户”用例，即“取款”（Withdraw Funds）、“查询账户”（Query Account）和“转账”（Transfer Funds），每个用例对应一种交易类型。现在考虑“取款”用例。该用例的主序列对应于客户成功取款的情况，其中包括读取 ATM 卡、验证客户 PIN 码、验证客户的账户中是否有足够的金额、（如果通过验证）提供现金、打印凭条和弹出 ATM 卡。

然而，通过比较三个用例可以看出每个用例的开始部分（即读取 ATM 卡和验证客户 PIN 码）对于所有这三个用例都是一样的。因此，我们可以将这三个用例的公共部分提取出来作为一个称为“验证 PIN 码”（Validate PIN）的包含用例。

这样的话，“取款”、“查询账户”和“转账”这三个用例就可以描述得更精简一点，即让它们作为包含“验证 PIN 码”用例的具体用例。图 21-1 表示了用例之间的关系。具体的“取款”用例首先从所包含的“验证 PIN 码”包含用例的描述开始，然后再继续“取款”用例的描述。具体的“转账”用例也是从“验证 PIN 码”包含用例的描述开始，然后再继续“转账”用例的描述。修改后的具体的“查询账户”用例也是如此。包含用例和具体用例会在下文阐述。

“验证 PIN 码”用例的主序列包括读取 ATM 卡、验证客户 PIN 码和 ATM 卡。如果验证通过，系统会提示客户选择一种交易：取款、查询或转账。其他分支则处理所有可能的错误情况，例如客户输入了错误的 PIN 码然后系统必须重新提示，或者 ATM 卡无法被识别或被挂失等。因为这些情况可以很容易地用可替换序列（alternative sequence）来描述，所以不需要将这些情况分成不同的扩展用例来描述。

372
>
373

21.2.1 “验证 PIN 码”用例

用例名：验证 PIN 码

概述：系统验证客户 PIN 码

参与者：ATM 客户

前置条件：ATM 机是空闲的，显示着欢迎信息

主序列：

1. 客户将 ATM 卡插入读卡器。
2. 如果系统成功识别该卡，那么读取该卡的卡号。
3. 系统提示客户输入 PIN 码。
4. 客户输入 PIN 码。
5. 系统检查卡的失效日期以及该卡是否已被挂失（遗失或被盗）。
6. 如果该卡未失效，系统检查用户输入的 PIN 码是否正确。
7. 如果 PIN 码正确，系统检查客户的哪些账户可以用该 ATM 卡访问。
8. 系统显示可访问的客户账户，并提示用户选择交易类型：取款、查询或转账。

可替换序列：

步骤 2：如果系统未能识别该卡，那么系统弹出该卡。

步骤 5：如果系统确定该卡失效，那么系统没收该卡。

步骤 5：如果系统确定该卡已被挂失（遗失或被盗），那么系统没收该卡。

步骤 7：如果客户输入的 PIN 码不正确，那么系统提示用户重新输入 PIN 码。

步骤 7：如果客户输错 PIN 码三次，那么系统没收该卡。

步骤 4 ~ 8：如果用户选择“取消”选项，那么系统取消交易并弹出 ATM 卡。

后置条件：客户的 PIN 码已被验证。

21.2.2 具体的“取款”用例

用例名：取款

概述：客户从一个有效账户中取出一定量的现金

参与者：ATM 客户

依赖：包含了“验证 PIN 码”（Validate PIN）用例

前置条件：ATM 机是空闲的，显示着欢迎信息

主序列：

1. 包含“验证 PIN 码”用例的步骤。
2. 客户选择“取款”选项，输入取款金额并选择账户号。
3. 系统检查客户在该账户中的余额是否足够以及用户所输入的取款金额是否超过取款上限。
4. 如果通过所有上述检查，系统授权通过本次取款请求。
5. 系统分发相应数额的现金。
6. 系统打印凭条，显示交易号、交易类型、取款金额和账户余额信息。
7. 系统弹出 ATM 卡。
8. 系统显示欢迎信息。

可替换序列：

步骤 3：如果系统确定所选账户号无效，那么系统显示错误信息并弹出 ATM 卡。

步骤 3：如果系统确定客户账户上没有足够的金额，那么系统显示道歉信息并弹出 ATM 卡。

步骤 3：如果系统确定取款金额超过了每日取款上限，那么系统显示道歉信息并弹出 ATM 卡。

步骤 5：如果 ATM 机现金不够，那么系统显示道歉信息、弹出 ATM 卡并关闭机器。

后置条件：客户账户的金额已被扣除。

21.2.3 具体的“查询账户”用例

用例名：查询账户

参与者：ATM 客户

概述：客户得知一个有效银行账户的余额

依赖：包含了“验证 PIN 码”（Validate PIN）用例

前置条件：ATM 机是空闲的，显示着欢迎信息

主序列：

1. 包含“验证 PIN 码”用例的步骤。
2. 客户选择“查询”选项，并输入账户号码。
3. 系统读取账户余额。
4. 系统打印凭条，显示交易号、交易类型和账户余额信息。
5. 系统弹出 ATM 卡。
6. 系统显示欢迎信息。

可替换序列：

步骤 3：如果系统确定所选账户号无效，那么系统显示错误信息并弹出 ATM 卡。

后置条件：客户查询了账户。

21.2.4 具体的“转账”用例

用例名：转账

概述：客户将一定数额的资金从一个有效账户转到另一个有效账户

参与者：ATM 客户

依赖：包含了“验证 PIN 码”（Validate PIN）用例

前置条件：ATM 机是空闲的，显示着欢迎信息

主序列：

1. 包含“验证 PIN 码”用例的步骤。
2. 客户选择“转账”选项并输入转账“金额”（amount）、“转出账户”（from account）和“转入账户”（to account）。
3. 如果系统确定“转出账户”里有足够的金额，那么系统进行转账。
4. 系统打印凭条，显示交易号、交易类型、转账金额和账户余额信息。
5. 系统弹出 ATM 卡。
6. 系统显示欢迎信息。

可替换序列：

步骤 3：如果系统确定“转出账户”号码无效，那么系统显示错误信息并弹出 ATM 卡。

步骤 3：如果系统确定“转入账户”号码无效，那么系统显示错误信息并弹出 ATM 卡。

步骤 3：如果系统确定客户的“转出账户”上没有足够的金额，那么系统显示道歉信息并弹出 ATM 卡。

后置条件：客户资金从指定的“转出账户”转到指定的“转入账户”。

21.3 静态建模

本节首先对问题域和系统上下文进行了考虑，然后讨论实体类的静态建模。可同时参阅第 7 章，其中用“银行系统”的例子详细介绍了静态建模。

21.3.1 问题域的静态建模

图 21-2 中的类图描述了问题域中的概念静态模型。一个银行有若干 ATM 机。每个 ATM 机被建模为一个复合类，其中包含“读卡器”（Card Reader）、一个“吐钞器”（Cash Dispenser）、一个“凭条打印机”（Receipt Printer）和一个用来和用户交互的“ATM 客户键盘 / 显示器”（ATM Customer Keyboard Display）。“ATM 客户”参与者将卡插入“读卡器”并通过“ATM 客户键盘 / 显示器”对系统的提示做出响应。“凭条打印机”为“ATM 客户”参与者打印凭条。此外，“ATM 操作员”参与者是专门维护 ATM 机的人员。

[376]

21.3.2 系统上下文的静态建模

软件系统的上下文类图使用静态建模表示法，将“银行系统”看做一个聚合类，描述了与“银行系统”存在交互关系的外部类。正如第 7 章所述，我们通过考虑问题域的静态建模过程中所确定的物理类来开发上下文类图。

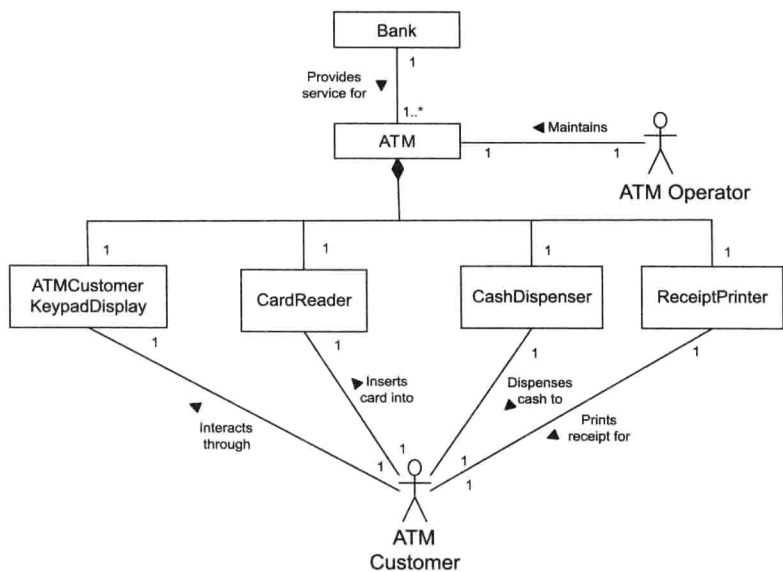


图 21-2 问题域的概念静态模型

如图 7-19 所示，从整个系统（包括硬件和软件）的角度看，“ATM 客户”和“ATM 操作员”参与者都位于系统外部。“ATM 操作员”通过键盘 / 显示器与系统交互。“ATM 客户”参与者通过读卡器、吐钞器、凭条打印机和 ATM 客户键盘 / 显示器等输入 / 输出（I/O）设备与系统交互。从整个软硬件系统的角度来说，这些输入 / 输出设备是系统的一部分。从软件的角度来说，这些输入 / 输出设备位于软件系统的外部。在软件系统的上下文类图中，输入 / 输出设备则被建模为外部类，如图 21-3 所示。

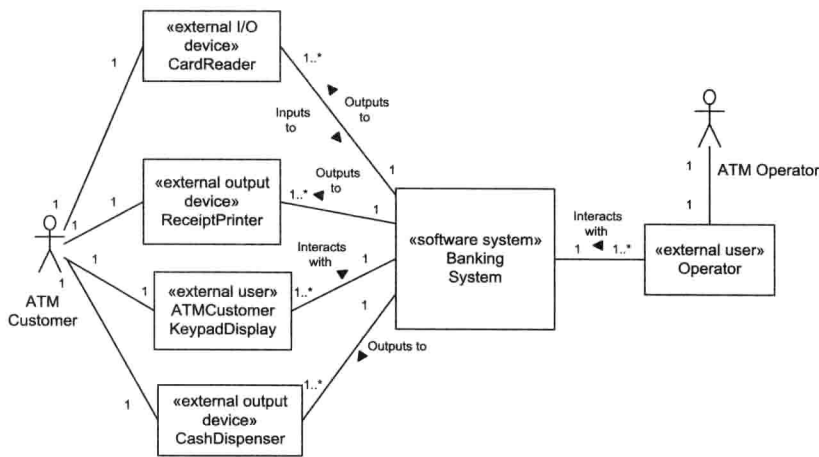


图 21-3 “银行系统”软件上下文类图

“ATM 客户”参与者使用的四个外部类是“读卡器”、“吐钞器”、“凭条打印机”和“ATM 客户键盘 / 显示器”；“ATM 操作员”通过键盘 / 显示器与系统交互。如第 7 章所述，“ATM 客户”和“ATM 操作员”都被建模为外部用户。ATM 机的每一个外部类都有一个实例。“银行系统”（见图 21-3）的软件系统上下文类图把软件系统刻画为一个聚合类，该类从外部类接收消息同时也向它们发送消息。

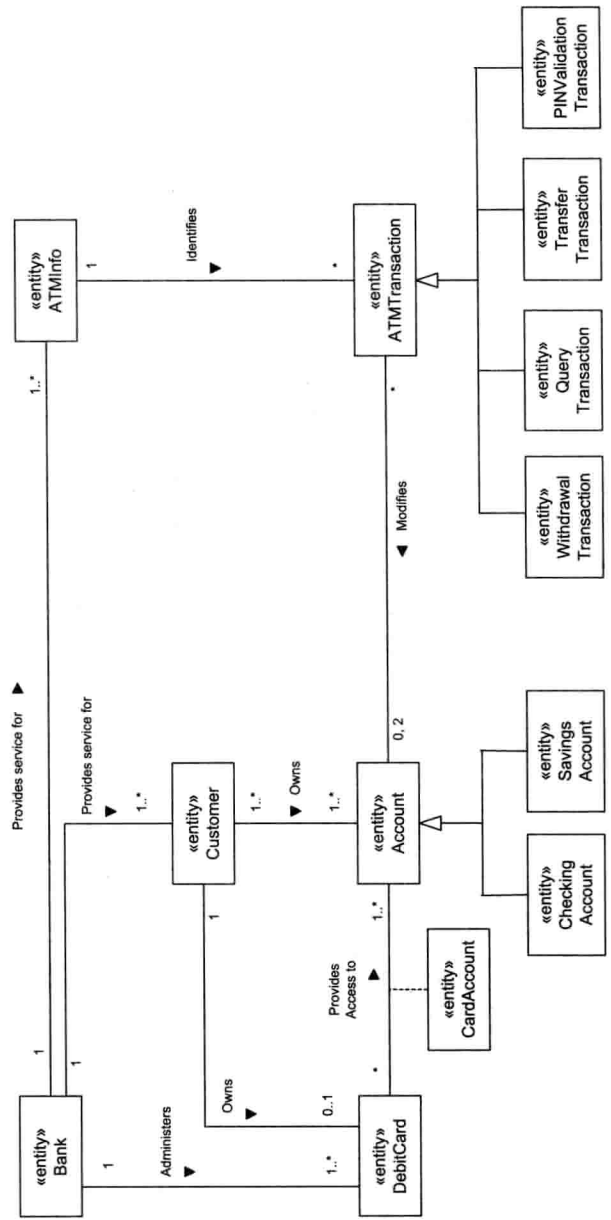


图21-4 “银行系统”的概念静态模型：实体类

21.3.3 实体类的静态建模

图 21-4 描述了实体类的静态模型，即实体类模型。图 21-5、图 21-6 和图 21-7 给出了每个实体类的属性。

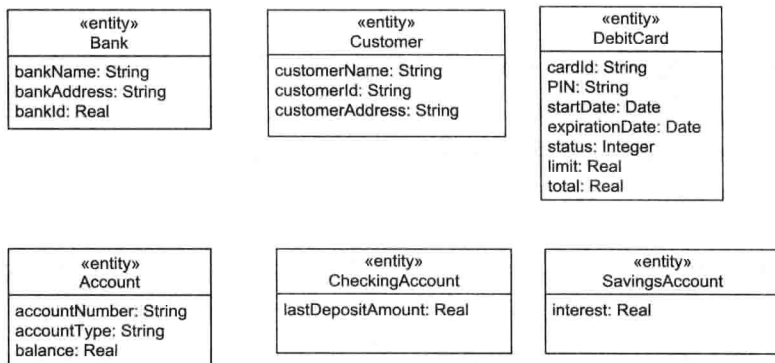


图 21-5 “银行系统”的概念静态模型：类属性

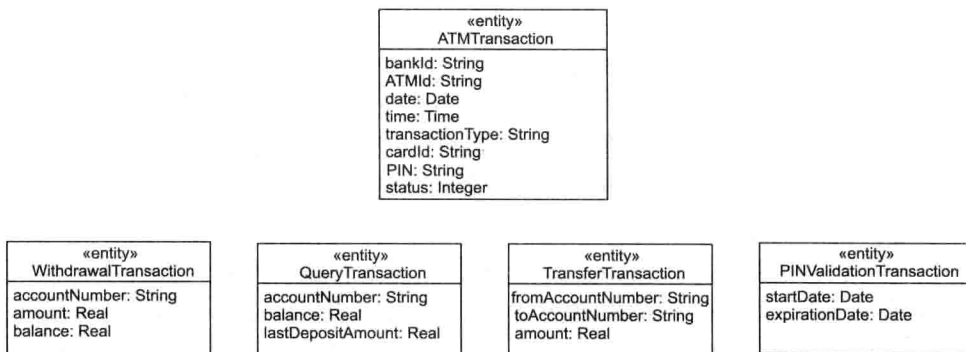


图 21-6 “银行系统”的概念静态模型：类属性（续）

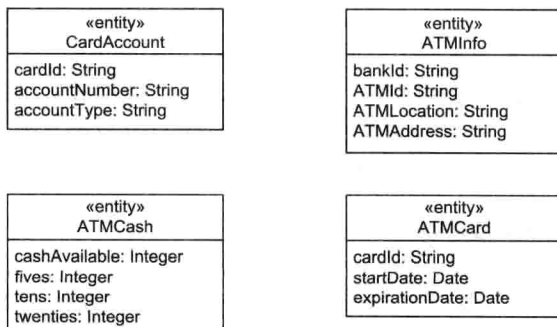


图 21-7 “银行系统”的概念静态模型：类属性（续）

图 21-4 描述了“银行”（Bank）实体类，它和“客户”（Customer）类以及“借记卡”（Debit Card）类之间存在一对多的关系。该实体类比较特别，因为它只有一个实例；它的属性包括“银行名称”（bank Name）、“银行地址”（bank Address）和“银行号”（bank Id）。“客户”类和“账户”（Account）类存在多对多的关系。由于存在支票账户（Checking Account）

和储蓄账户 (Savings Account) 且两者有部分公共属性, 因此“账户”类可以被特化为“支票账户”类和“储蓄账户”类。这样两个子类可以共享一些属性, 即“账户号” (account Number)、 “账户类型” (account Type) 和“余额” (balance)。其他属性则为“支票账户” (如最后一次“存款金额” (Deposit Amount)) 和“储蓄账户” (如累计的“利息” (interest)) 所专有。

“ATM 交易” (ATM Transaction) 类可以修改“账户”类, 而“ATM 交易”类又能被进一步特化为各种不同类型的交易类, 包括“取款交易” (Withdrawal Transaction) 类、“查询交易” (Query Transaction) 类、“转账交易” (Transfer Transaction) 类或“PIN 码验证交易” (PIN Validation Transaction) 类。“ATM 交易”父类包含交易类的共有属性, 包括“交易号” (transaction Id) (该属性实际上是由“银行号”、“ATM 号”、“日期”和“时间”这些属性拼接而成)、“交易类型” (transaction Type)、“卡号” (card Id)、“PIN 码” (PIN) 和“状态” (status), 其他属性则为特定类型的交易类所专有。因此, 对于“取款交易”子类, 它所专有的属性包括“账户号” (account Number)、“金额” (amount) 和“余额” (balance)。对于“转账交易”子类, 它所专有的属性是“转出账户号” (from Account Number) (可以是支票账户或储蓄账户)、“转入账户号” (to from Account Number) (可以是支票账户或储蓄账户) 以及“金额” (amount)。

还有一个“卡账户” (Card Account) 关联类。这种类用于所要描述的属性属于关联关系而非关联关系所连接的类的情况。这样, 在“借记卡”类和“账户”类之间的多对多关联中, 能被某张借记卡所访问的个人账户应该是“卡账户” (Card Account) 这一关联类的属性, 而非“借记卡”类或“账户”类的属性。卡账户类的属性是卡号 (Card Id)、账户号 (account Number) 和账户类型 (account Type)。

此外, 也需要用实体类建模 21.2 节中描述的其他信息。例如, 用“ATM 卡” (ATM Card) 类来表示塑料卡的磁条上所读出的信息; 用“ATM 现金” (ATM Cash) 类表示在 ATM 机中 5 美元、10 美元或 20 美元面额的现金; 用“凭条” (Receipt) 类表示关于一个交易的信息, 但由于它和前面描述的交易类的信息是一样的, 因此就没有必要单独表示为一个类了。

21.4 对象组织

我们接下来考虑将系统分解为对象, 为后面定义动态模型做好准备。对象组织准则可以帮助我们确定系统中的对象。在类和对象确定之后, 我们会为每一个用例开发通信图或顺序图来表示用例中的对象以及它们之间的动态交互顺序。

21.4.1 客户端 / 服务器子系统组织

由于“银行系统”是一个客户端 / 服务器应用, 一些对象属于 ATM 客户端而另一些对象则属于银行服务, 因此我们从识别作为聚合或组合对象的子系统开始。在客户端 / 服务器系统中, 子系统很好确定。这样, 在“银行系统”中有一个客户端子系统称为“ATM 客户端子系统” (ATM Client Subsystem), 每个 ATM 机上运行着它的一个实例。还有一个服务子系统称为“银行服务子系统” (Banking Service Subsystem), 它只有一个实例 (图 21-8)。这是一个基于地理分布的子系统组织的例子, 其中的地理分布已经在问题描述中明确了。“银行服务子系统”和“ATM 客户端子系统”这两个子系统都被描述为聚合类, 并且二者之间存在一对

多关联。所有的外部类都和“ATM 客户端子系统”进行交互和通信。

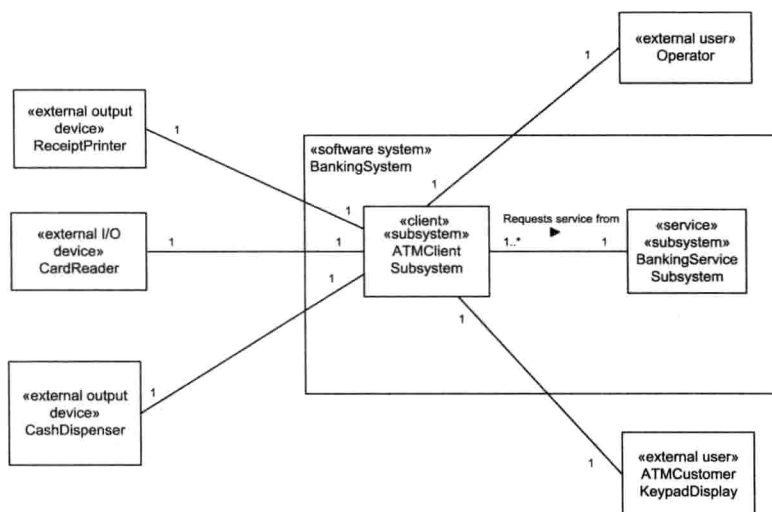


图 21-8 银行系统：主要子系统

21.4.2 ATM 客户端对象和类的组织：边界对象

下一步是要确定“ATM 客户端”中的对象和类。首先考虑边界对象和边界类。边界类可以基于软件系统的上下文图来确定，如图 21-9 所示，该图将“银行系统”显示为一个聚合类。

我们为每个外部类设计一个边界类。输入/输出设备类包括：“读卡器接口”（Card Reader Interface），ATM 卡是通过这个接口读取的；“吐钞器接口”（Cash Dispenser Interface），用来分发现金；“凭条打印机接口”（Receipt Printer Interface），用于打印凭条。此外还有“客户交互”（Customer Interaction）类，这是一个通过键盘/显示器和客户交互的用户交互类，能够显示文本消息、提示客户进行操作并接收客户输入。“操作员接口”（Operator Interaction）类是一个和 ATM 操作员（为 ATM 机补充现金）交互的用户交互类。一个 ATM 机的每个边界类都只有一个实例。

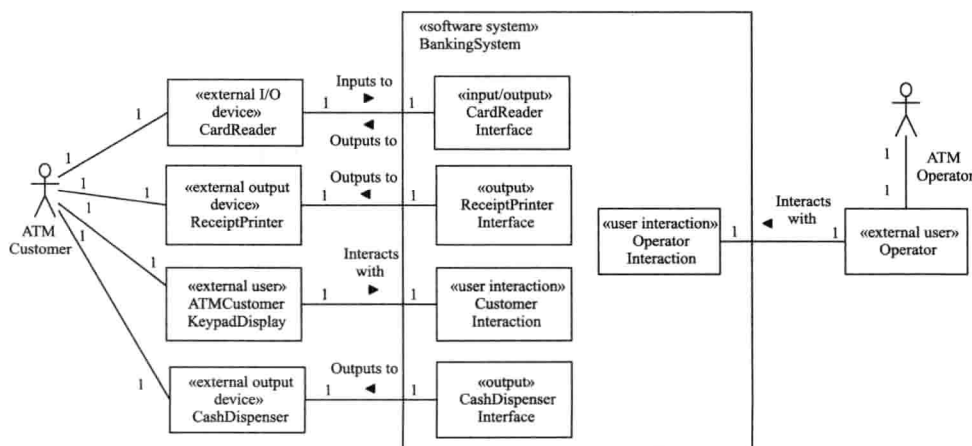


图 21-9 “银行系统”的外部类和边界类

21.4.3 ATM 客户端对象和类的组织：用例中的对象

接下来，我们考虑单个用例以及参与用例的对象。首先，考虑“验证 PIN 码”这一包含用例，该用例描述了客户如何将 ATM 卡插入读卡器、系统如何提示客户输入 PIN 码以及系统如何检查客户输入的 PIN 码是否正确。该用例中，我们首先确认“读卡器接口”对象是否需要读取 ATM 卡。由于需要保存从 ATM 卡上读取的信息，所以我们可以确定这里需要一个实体对象来存储“ATM 卡”信息。“客户交互”对象用来通过键盘 / 显示器与客户交互，在这里，该对象用来提示客户输入 PIN 码。发送给“银行服务子系统”用来验证 PIN 码的信息被保存在一个“ATM 交易”对象中。为了验证 PIN 码，交易信息需要包含 PIN 码以及 ATM 卡号。为了保证 ATM 机中各活动执行的顺序，我们确定还需要一个控制对象，即“ATM 控制”（ATM Control）。

下面考虑“取款”用例，如果 PIN 码有效且客户选择了取款选项则会进入该用例。在该用例中，客户输入取款金额以及借记卡号，然后系统验证是否能授权该取款操作。如果验证通过，系统分发现金、打印凭条并弹出 ATM 卡。该用例需要额外的对象。客户取款的信息（包括账户号及取款金额）需要存储在“ATM 交易”对象中。为了分发现金，需要一个“吐钞器接口”对象。我们还需要维护 ATM 机中现金的数量，因此确定需要一个称作“ATM 现金”（ATM Cash）的实体对象，该对象的值随着每次取款而相应减少。最后，需要一个“凭条打印机接口”对象来打印凭条。和前面一样，“ATM 控制”对象控制该用例的执行序列。

检查完其他用例，我们发现还需要一个额外的对象，即“操作员交互”（Operator Interaction）对象，该对象用于所有由 ATM “操作员参与者”发起的用例。“操作员交互”对象需要向“ATM 控制”发送“启动”（startup）和“关闭”（shutdown）事件，这是因为操作员维护 ATM 机和客户使用 ATM 机这两件事情不能同时进行。

基于以上分析，我们可以得到如图 21-10 所示的“ATM 客户端子系统”中的类，这些类都被表示为聚合类。除了图 21-9 中的三个输入 / 输出设备类和两个用户交互类，还有三个实体类和一个状态相关的控制类。

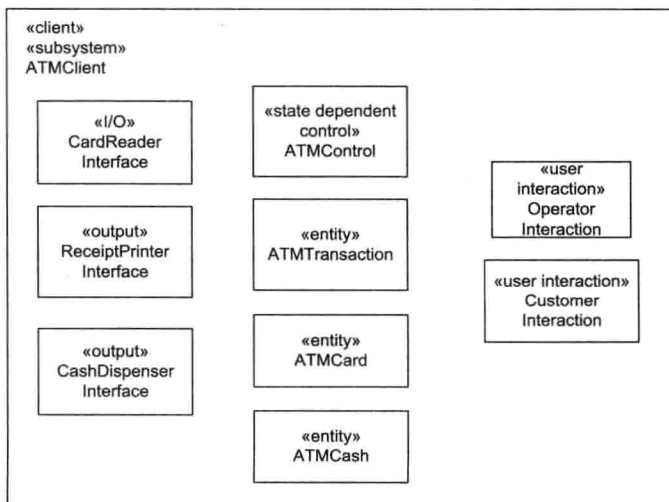


图 21-10 ATM 客户端子系统类

21.4.4 服务子系统对象组织

有些实体对象的作用范围涉及整个银行，所有的 ATM 机都需要访问这些对象。因此，这些对象需要保存在服务器端的“银行服务子系统”中。这些对象包括含有银行客户信息的“客户”（Customer）对象、含有各个银行账户信息的“账户”（Account）对象（包括支票账户和储蓄账户）以及包含银行所维护的所有借记卡信息的“借记卡”（Debit Card）对象。这些对象所对应的类都包含在图 21-4 所描述的实体类的静态模型中。

“银行服务子系统”中的实体类包括“客户”超类、“账户”超类、“支票账户”子类、“储蓄账户”子类以及“借记卡”类。还有一个“ATM 交易”对象，该对象需要从客户端发送到服务器。客户端向“银行服务”发送交易请求，然后“银行服务”向客户端返回一个响应。交易信息作为一个实体对象以“交易日志”（Transaction Log）的形式保存在服务器端，从而记录交易历史。作为 ATM 交易消息一部分的临时交易数据可能与持久化的交易数据有所不同，例如交易状态在交易结束之后是已知的，但在交易过程中却是未知的。

为了处理客户请求，服务器端还需要业务逻辑对象来定义特定业务的应用逻辑。具体而言，每一种 ATM 交易类型都需要一个交易管理器类来定义处理交易的业务规则。业务逻辑对象包括“PIN 码验证交易管理器”（PIN Validation Transaction Manager）、“取款交易管理器”（Withdrawal Transaction Manager）、“查询交易管理器”（Query Transaction Manager）和“转账交易管理器”（Transfer Transaction Manager）。例如，在“取款交易管理器”类中定义的业务规则是：（1）账户在取款之后余额必须大于等于零；（2）取款金额不能超过每日取款上限，上限的值由“借记卡”实体类的“限额”（limit）属性给出。

21.5 动态建模

动态模型描述了参与每个用例的对象之间的交互。开发动态模型的起点是用例以及对象结构组织过程中所确定的对象。满足每个用例需要的对象间消息通信序列可以用顺序图或通信图描述。通常只需要在两者中选择一个就足够了。在本例中，我们将客户端子系统的这两种图都画出来以便进行比较。

由于“银行系统”是一个客户端/服务器系统，因此在此之前已经确定将系统的结构分解为客户端和服务子系统，如图 21-8 所示。我们用通信图来表示客户端和服务子系统。

图 21-11 和图 21-16 中所描述的通信图是用来实现 ATM 客户端“验证 PIN 码”用例和“取款”用例的。通信图也可以用来实现 ATM 客户端的“转账”用例、“查询账户”用例以及由操作员触发的各个用例。

ATM 客户端的“验证 PIN 码”和“取款”的通信图是状态相关的。“ATM 控制”对象定义了交互中状态相关的部分，这一部分执行了 ATM 状态图。我们用状态相关的动态分析方法来确定对象之间是如何交互的。图 21-13 和图 21-18 中的状态图分别展示了这两种用例。21.5.1 节和 21.5.3 节分别介绍了这两个客户端用例的动态分析。

银行服务按照收到请求的先后顺序处理来自多个 ATM 机的交易。每个交易的处理都是完整和自包含的，所以用例中的银行服务部分不是状态相关的。因此，这些用例需要一个无状态的动态分析。服务器端“验证 PIN 码”和“取款”用例的通信图在图 21-14 和图 21-19 中给出。这两个服务器端用例的动态分析分别在 21.5.2 节和 21.5.4 节中给出。

考虑对象之间是如何交互的。我们针对“验证 PIN 码”和“取款”用例给出了详细

的例子。在客户端，给出了通信图和顺序图。在两种图中，消息序列的编号和描述都是一样的。

21.5.1 客户端验证 PIN 码交互图的消息序列描述

客户端的“验证 PIN 码”交互图起始于客户将 ATM 卡插入读卡器。该消息的顺序编号起始于 1，这是由参与者发起的第一个外部事件。表示到达系统中软件对象的消息的后续编号分别为 1.1、1.2、1.3，最后结束于 1.4，表示系统显示给参与者的响应。参与者的下一个输入是编号为 2 的外部事件，随后是内部事件 2.1、2.2 等。后续的消息序列描述对应于图 21-11 中的通信图以及图 21-12 中的顺序图。

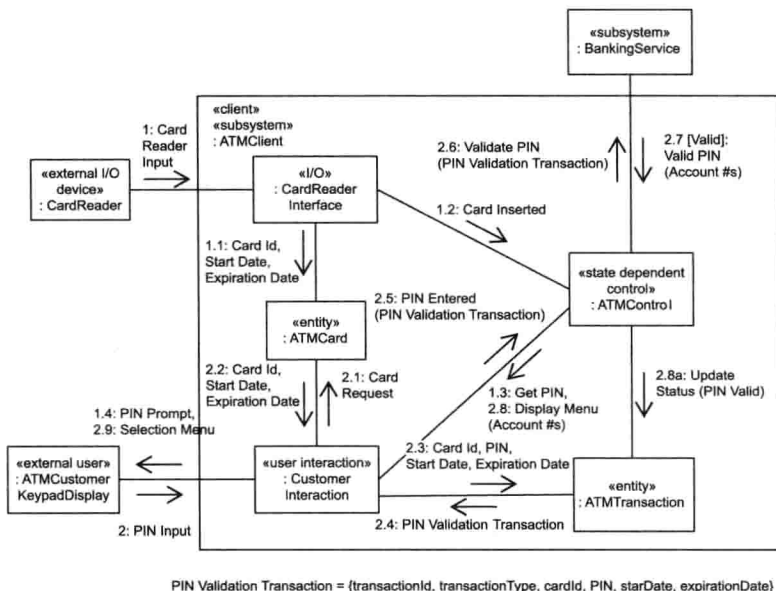


图 21-11 通信图：ATM 客户端“验证 PIN 码”用例

由于“验证 PIN 码”交互图是状态相关的，因此有必要考虑由“ATM 控制”对象执行的 ATM 状态图。具体而言，需要考虑状态图（见图 21-13）和“ATM 控制”对象（见通信图）之间的交互。下面的消息序列阐述了该状态图的状态和转移，对应于图 21-11 中的通信图和图 21-12 中的顺序图中的事件。消息序列描述如下：

385

- 1: “ATM 客户”参与者将 ATM 卡插入“读卡器”。“读卡器接口”对象读取卡中信息。
- 1.1: “读卡器接口”将 ATM 卡输入数据（包含了“卡号”、“生效日期”、“失效日期”）发送给“ATM 卡”实体对象。
- 1.2: “读卡器接口”将“卡片插入”（Card Inserted）的消息发送给“ATM 控制”对象。“卡片插入”事件使得“ATM 控制”状态图从空闲状态（初始状态）转移到“等待 PIN 码输入”（Waiting for PIN）状态。与该转移关联的输出事件是“获取 PIN 码”（Get PIN）。
- 1.3: “ATM 控制”将“获取 PIN 码”消息发送给“客户交互”对象。
- 1.4: “客户交互”对象将“PIN 码提示”显示给“ATM 客户”参与者。

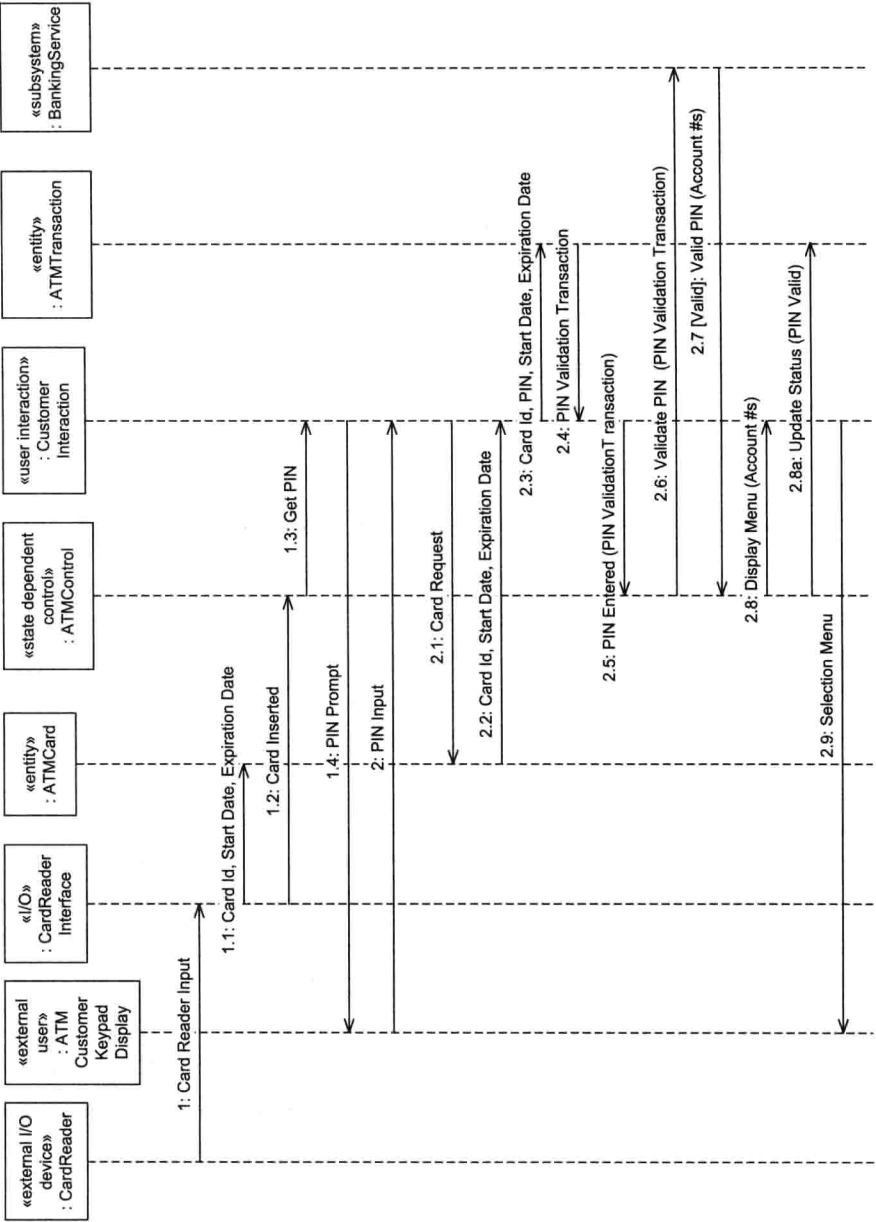


图21-12 顺序图：ATM客户端“验证PIN码”用例

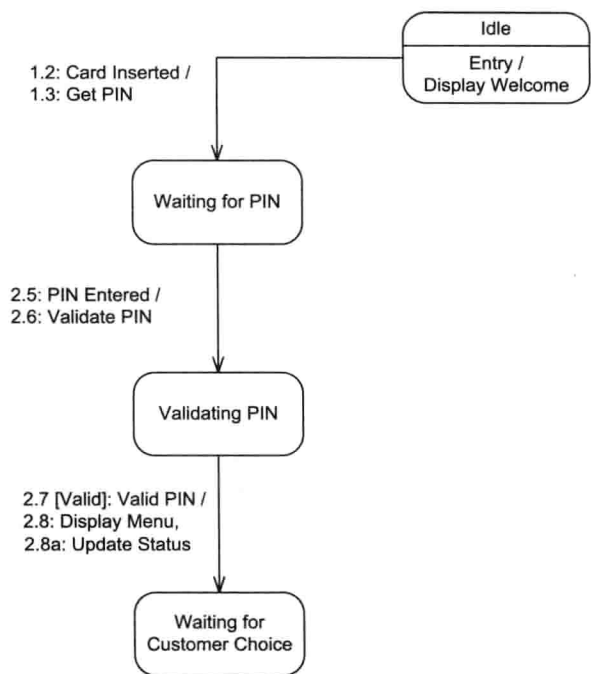


图 21-13 “ATM 控制”的状态图：“验证 PIN 码”用例

- 2: “ATM 客户”向“客户交互”对象输入 PIN 码。
- 2.1: “客户交互”对象向“ATM 卡”请求卡数据。
- 2.2: “ATM 卡”向“客户交互”对象提供卡数据。
- 2.3: “客户交互”向“ATM 交易”实体对象发送“卡号”、“PIN 码”、“生效日期”和“失效日期”。
- 2.4: “ATM 交易”实体对象向“客户交互”发送“PIN 码验证交易”。
- 2.5: “客户交互”对象向“ATM 控制”发送“PIN 码已输入”(PIN Entered) (“PIN 码验证交易”) 的消息。“PIN 码已输入”事件使得“ATM 控制”状态图从“等待 PIN 码输入”状态转移到“验证 PIN 码”状态。与该转移关联的输出事件是“验证 PIN 码”。
- 2.6: “ATM 控制”向“银行服务”发送一个“验证 PIN 码”(“PIN 码验证交易”) 的请求。
- 2.7: “银行服务”验证 PIN 码并向“ATM 控制”发送“有效 PIN 码”(Valid PIN) 的响应。然后，“ATM 控制”转移到“等待客户选择”(Waiting for Customer Choice) 状态。该转移的输出事件是与“ATM 控制”对象发送的输出消息相对应的“显示菜单”(Display Menu) 和“更新状态”(Update Status)。
- 2.8: “ATM 控制”对象向“客户交互”对象发送“显示菜单”消息。
- 2.8a: “ATM 控制”对象向“ATM 交易”对象发送“更新状态”消息。
- 2.9: “客户交互”向“ATM 客户”显示包含“取款”、“查询”和“转账”选项的菜单。

与“验证 PIN 码”用例中的可替换场景序列相对应的可替换场景的动态建模在第 11 章中进行了描述。可替换场景是基于交互图和状态图描述的。

21.5.2 服务器端验证 PIN 码交互图的消息序列描述

现在考虑服务器端“验证 PIN 码”包含用例的交互图。“借记卡”实体对象包含了银行中所有借记卡的相关信息，在服务器验证 PIN 码时需要访问该对象。一张借记卡可访问多个账户，如果 PIN 码验证通过，需要访问“卡账户”实体对象来获得这些账户的账户号。

此外，每个交易都有一个**业务逻辑对象**，该对象封装了管理交易执行的业务应用逻辑。业务逻辑对象从客户端的“ATM 控制”对象接收交易请求，然后与实体对象交互以确定向“ATM 控制”对象返回什么样的响应。例如，“PIN 码验证”交易的业务逻辑对象是“PIN 码验证交易管理器”对象。

以下服务器端“验证 PIN 码”交互图的消息序列描述对应于图 21-14 的通信图和图 21-15 的顺序图。

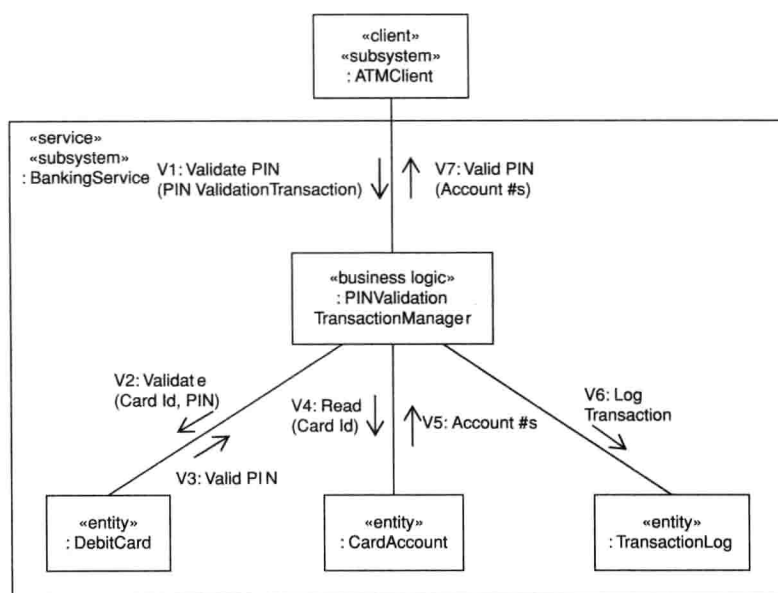


图 21-14 通信图：银行服务“验证 PIN 码”用例

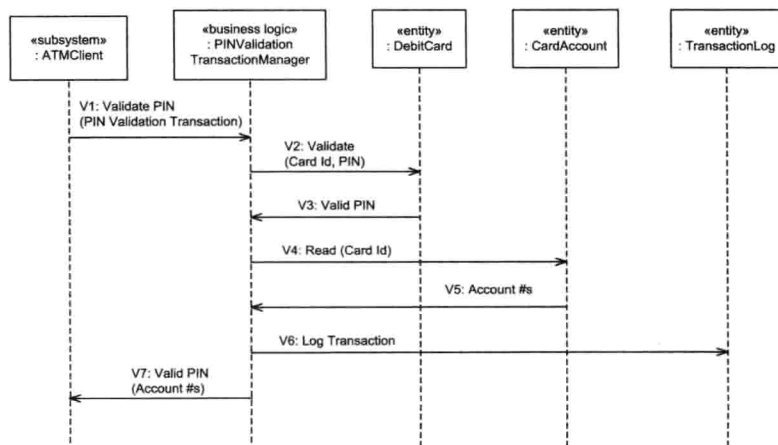


图 21-15 顺序图：银行服务“验证 PIN 码”用例

V1：“ATM 客户端”向“PIN 码验证交易管理器”发送“验证 PIN 码”请求。“PIN 码验证交易管理器”中包含判断 PIN 码是否有效的业务逻辑，即判断客户输入的 PIN 码是否与银行服务数据库中的 PIN 码相符。

V2：“PIN 码验证交易管理器”向“借记卡”实体对象发送一个“验证”(Validate)消息(包含“卡号”、“PIN”)，要求其根据卡号和客户输入的 PIN 码来验证客户的借记卡是否有效。

V3：“借记卡”检查验证客户输入的 PIN 码是否与“借记卡”中记录的 PIN 码符合、卡的状态是否正常(无挂失)以及该卡是否过期。如果所有验证通过，“借记卡”向“PIN 码验证交易管理器”发送一个有效 PIN 码的响应。

V4：如果验证通过，“PIN 码验证交易管理器”向“卡账户”实体对象发送消息，要求其返回当前卡号可访问的账户号。

V5：“卡账户”返回有效的账户号。

V6：“PIN 码验证交易管理器”用“交易日志”(Transaction Log)来记录交易日志。

V7：“PIN 码验证交易管理器”向“ATM 客户端”发送一个“有效 PIN 码”的响应。如果 PIN 码验证成功，还会发送账户号。

21.5.3 客户端取款交互图的消息序列描述

客户端“取款”交互图的消息序列描述了包括通信图(图 21-16)与顺序图(图 21-17)的消息，也描述了 ATM 状态图(图 21-18)上的相关状态和转移。下面的消息接着 21.5.1 节中客户端“验证 PIN 码”交互图的消息继续编号。

3：“ATM 客户”参与者向“客户交互”对象输入“取款”选项，并告知账户号(支票账户和储蓄账户)以及取款金额。

3.1：“客户交互”对象将客户选项发送给“ATM 交易”对象。

3.2：“ATM 交易”对象将“取款交易”详细信息返回给“客户交互”对象。“取款交易”包含了交易号、交易类型、卡号、PIN 码、账户号和金额。

3.3：“客户交互”对象将“选择取款”(Withdrawal Selected) (“取款交易”)的请求发送给“ATM 控制”对象。接着，“ATM 控制”对象转移到“处理取款”(Processing Withdrawal)状态，与该转移相关的输出事件是“请求取款”(Request Withdrawal)和“显示等待”(Display Wait)。

3.4：“ATM 控制”对象向“银行服务”发送一个包含“取款交易”(Withdrawal Transaction)的“取款请求交易”(Request Withdrawal)。

3.4a：“ATM 控制”对象向“客户交互”对象发送“显示等待”(Display Wait)的消息。

3.4a.1：“客户交互”对象向“ATM 客户”显示“等待提示”(Wait Prompt)。

3.5：“银行服务”向“ATM 控制”对象发送一个“允许取款”(Withdrawal Approved)响应(包含取款“金额”、“余额”信息)。该事件导致“ATM 控制”转移至“分发现金”(Dispensing)状态。同时，输出事件为“分发现金”(Dispense Cash)以及“更新状态”。

3.6：“ATM 控制”对象向“吐钞器接口”发送一个“分发现金”消息(包含取款“金额”信息)。

3.6a：“ATM 控制”对象向“ATM 交易”对象发送一个“更新状态”消息（包含取款“金额”、“余额”信息）。

3.7：“吐钞器接口”向“ATM 现金”（ATM Cash）发送一个“取款”消息（包含取款“金额”信息）。

3.8：“ATM 现金”向“吐钞器接口”发送一个表示接受的“现金响应”（Cash Response），明确待分发的每种面额钞票的张数。

3.9：为了把钞票分发给客户，“吐钞器接口”向“吐钞器”外部输出设备发送“提款机输出”（Dispenser Output）命令。

3.10：“吐钞器接口”向“ATM 控制”对象发送“现金已分发”（Cash Dispensed）的消息。该消息导致“ATM 控制”转移至“打印”（Printing）状态。与该转移相关的输出事件是“打印凭条”、“显示分发现金”（Display Cash Dispensed）以及“确认现金分发”（Confirm Cash Dispensed）。

3.11：“ATM 控制”对象向“凭条打印机接口”（Receipt Printer Interface）发送“打印凭条”（Print Receipt）消息。

3.11a：“ATM 控制”向“客户交互”发送“显示分发现金”消息。

3.11a.1：“客户交互”把“现金已分发”提示显示给“ATM 客户”。

3.11b：“ATM 控制”向“银行服务”发送一个“确认现金分发”消息。

3.12：“凭条打印机接口”向“ATM 交易”请求交易数据。

3.13：“ATM 交易”向“凭条打印机接口”发送交易数据。

3.14：“凭条打印机接口”向“凭条打印机”外部输出设备发送凭条“打印机输出”（Printer Output）消息。

3.15：“凭条打印机接口”向“ATM 控制”发送“凭条已打印”（Receipt Printed）消息。由此，“ATM 控制”转移到“正在弹出卡”（Ejecting）状态。此时，输出事件是“弹出卡”（Eject）。

3.16：“ATM 控制”向“读卡器接口”发送“弹出卡”消息。

3.17：“读卡器接口”向“读卡器”外部输入/输出设备发送“读卡器输出”（Card Reader Output）消息。

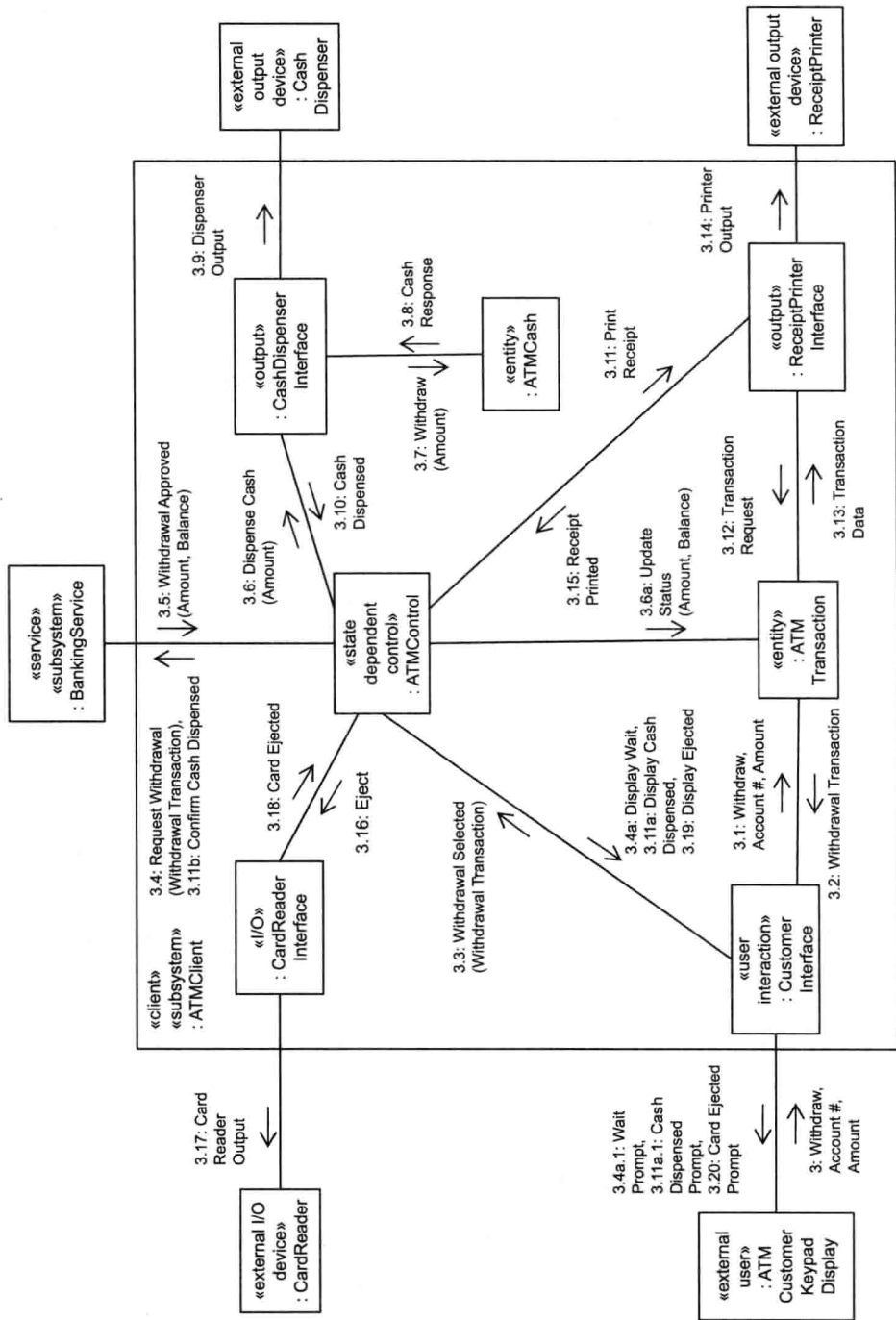
3.18：“读卡器接口”向“ATM 控制”对象发送“卡弹出”（Card Ejected）消息。“ATM 控制”转移到“已终止”（Terminated）状态。此时，输出事件是“显示卡已弹出”（Display Ejected）。

3.19：“ATM 控制”对象向“客户交互”对象发送“显示卡已弹出”的消息。

3.20：“客户交互”对象向“ATM 客户”显示“卡已弹出”的提示消息。

21.5.4 服务器端取款交互图的消息序列描述

参与服务器端“取款”用例的业务逻辑对象是“取款交易管理器”，该对象封装了确定客户是否可以从某个账户中取款的逻辑。其他参与服务器用例的业务逻辑对象包括“转账交易管理器”（封装了确定客户是否能够将资金从一个账户转到另一个账户的逻辑）和“查询交易管理器”。后者由于过于简单，所以严格来说未必有必要将其封装为一个独立的业务逻辑对象；其功能也可以通过“账户”对象的“读取”操作来实现。然而，为了与其他业务逻辑对象一致，此处仍然将其作为一个独立的业务逻辑对象。



Withdrawal Transaction = {transactionId, transactionType, cardId, PIN, account#, amount}

图21-16 通信图：ATM客户端“取款”用例

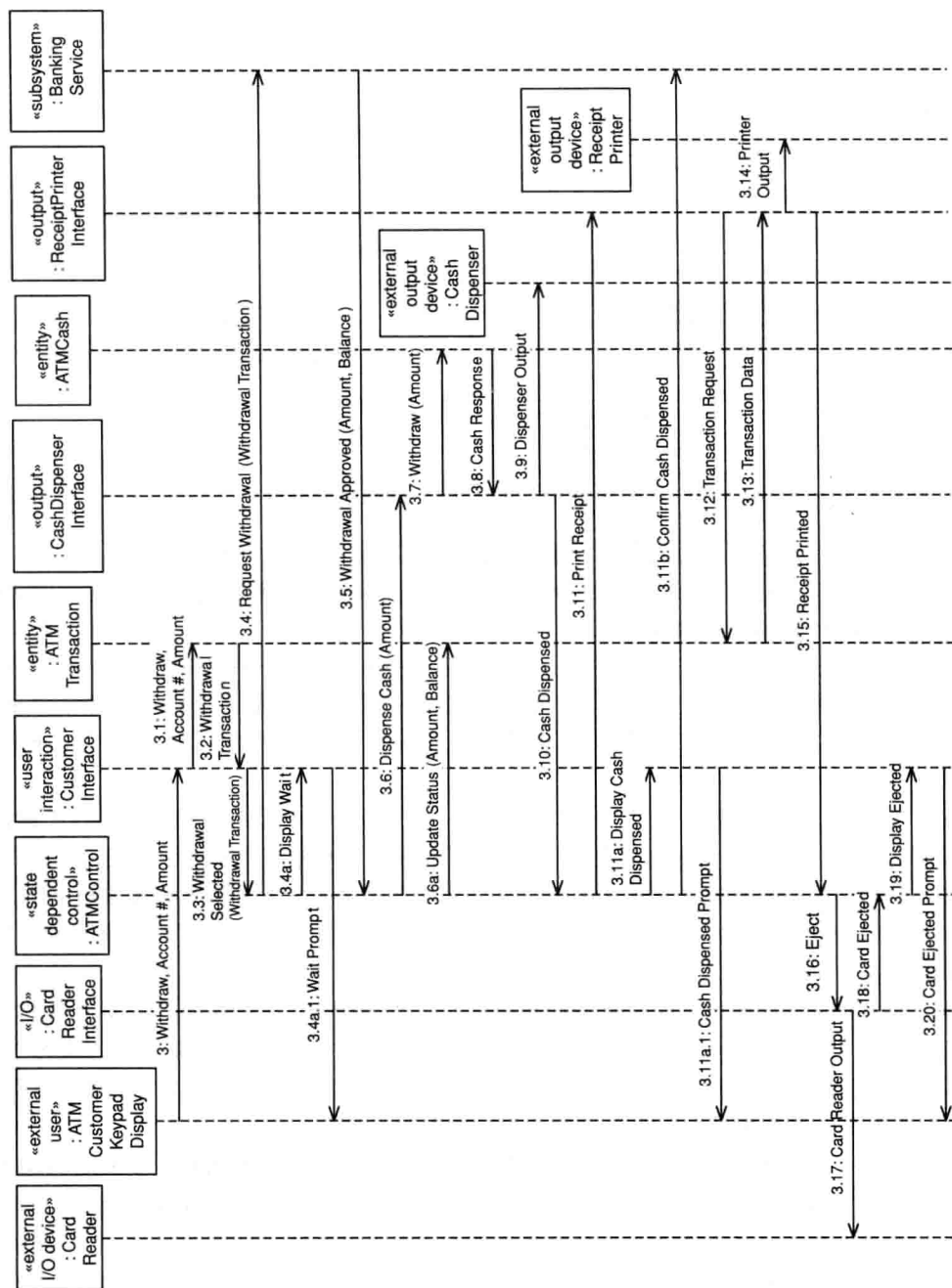


图21-17 顺序图：ATM客户端“取款”用例

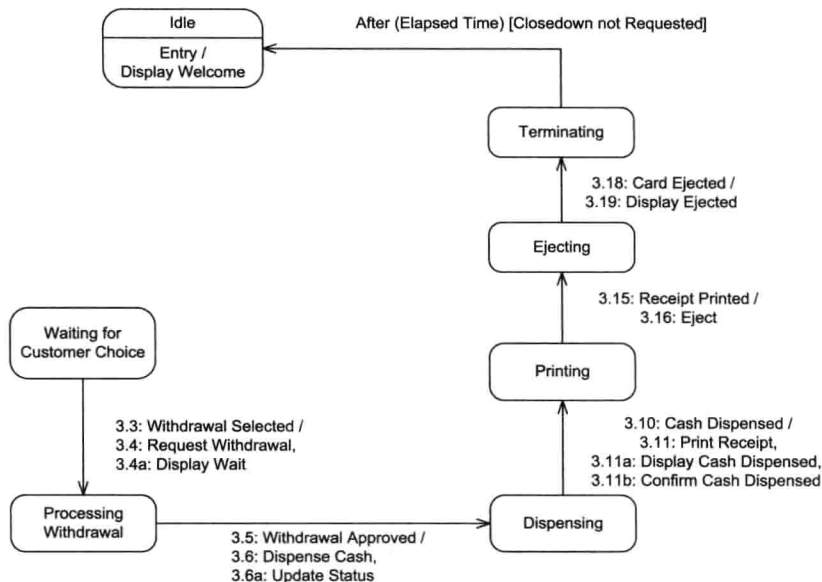


图 21-18 “ATM 控制” 的状态图：“取款” 用例

我们对服务器端“取款”用例做了详细分析。服务器端“转账”用例和“查询账户”用例也需要类似的分析方法。下面的消息序列描述与图 21-19 中的服务器端的“取款”用例的通信图和图 21-20 中的顺序图相对应。

W1：“ATM 客户端”向“取款交易管理器”（其中包含了确定是否允许取款业务逻辑）发送“取款请求”。传入的取款交易包括“交易号”、“交易类型”、“卡号”、“PIN 码”、“账户号”以及取款“金额”。

W2：“取款交易管理器”向“借记卡”发送一个“检查每日取款上限”（Check Daily Limit）消息（包含“卡号”和取款“金额”信息）。“借记卡”检查该卡号是否已经达到每日取款上限。“借记卡”允许取款的条件为：当日已取款总额 + 本次取款金额 ≤ 每日取款上限。

W3：“借记卡”向“取款交易管理器”响应一个表示接受或拒绝的“每日上限响应”（Daily Limit Response）。

W4：如果接受了请求，“取款交易管理器”向“账户对象”（该对象为“支票账户”或“储蓄账户”的一个实例）发送一条消息，该消息可以在客户账户余额足够的情况下取出相应的金额。“账户”对象允许取款的条件为：账户余额 - 请求取款金额 ≥ 0。如果金额足够，“账户”对象就从“余额”中减去“请求取款金额”。

W5：“账户”对象向“取款交易管理器”返回“允许取款”（Withdrawal Approved）消息（包括取款“金额”和“余额”信息）或者“拒绝取款”（Withdrawal Denied）消息。

W6：如果从账户中成功取款，“取款交易管理器”则向“借记卡”发送一个“更新每日取款总额”（Update Daily Total）的消息（包含“卡号”和“金额”信息），由此可把请求的取款金额加入当日取款总额。

W7：“取款交易管理器”将“交易日志”记入本次交易信息。

W8：“取款交易管理器”向“ATM 客户端”返回“允许取款”消息（包括取款“金额”和“余额”信息）或者“拒绝取款”消息。

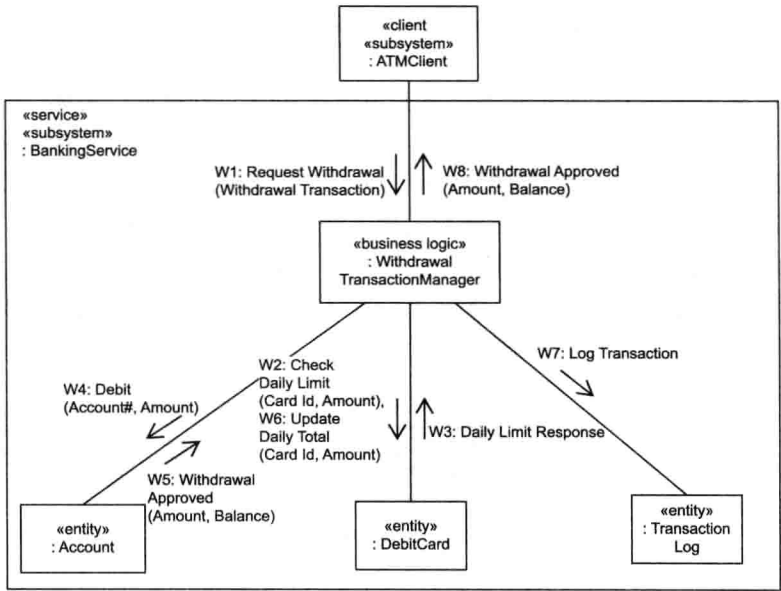


图 21-19 通信图：银行服务的“取款”用例

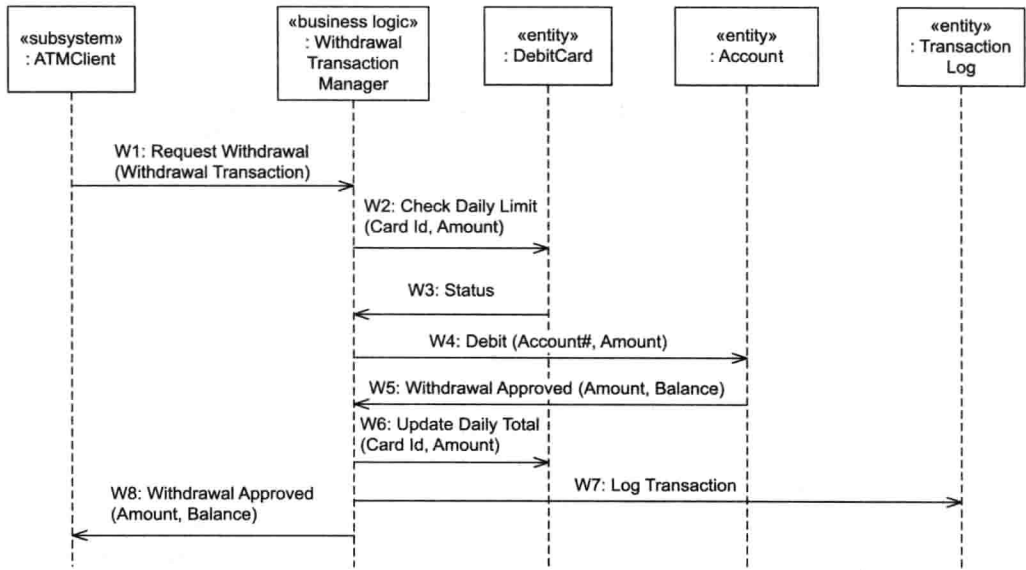


图 21-20 顺序图：银行服务的“取款”用例

21.6 ATM 状态图

由于“ATM 控制”（ATM Control）是一个控制对象，因此需要为它定义一个状态图。在图 21-14 和图 21-18 中分别描述了对应于“验证 PIN 码”和“取款”用例的部分状态图。这里也有必要为其他用例画状态图，并为这些用例的可替换场景定义状态和转移，由此来覆盖异常情况。我们一开始用扁平状态图来描述这些用例。在第 10 章中阐述了如何将状态图集成到不同的用例中以及如何设计具有层次结构的 ATM 控制状态图。具有层次结构的状态图的一

大好处就是可以像图 21-21 ~ 图 21-24 中那样分阶段表示状态。这些图上的事件序列编号对应于此此前描述的对象交互。

图 21-21 中描述了一个由 5 个状态组成的顶层状态图：初始的“关闭”（Closed Down）状态、“空闲”（Idle）状态以及 3 个复合状态，即“处理客户输入”（Processing Customer Input）、“处理交易”（Processing Transaction）和“结束交易”（Terminating Transaction）。每个复合状态可以进一步分解为更细粒度的状态图，如图 21-22、图 21-23 和图 21-24 所示。

在系统初始化时，“启动”事件使得 ATM 机从初始的“关闭”状态转移到“空闲”状态。进入“空闲”状态的时候会触发“显示欢迎”（Display Welcome）事件。处于“空闲”状态的时候，ATM 机一直等待由客户触发的事件。

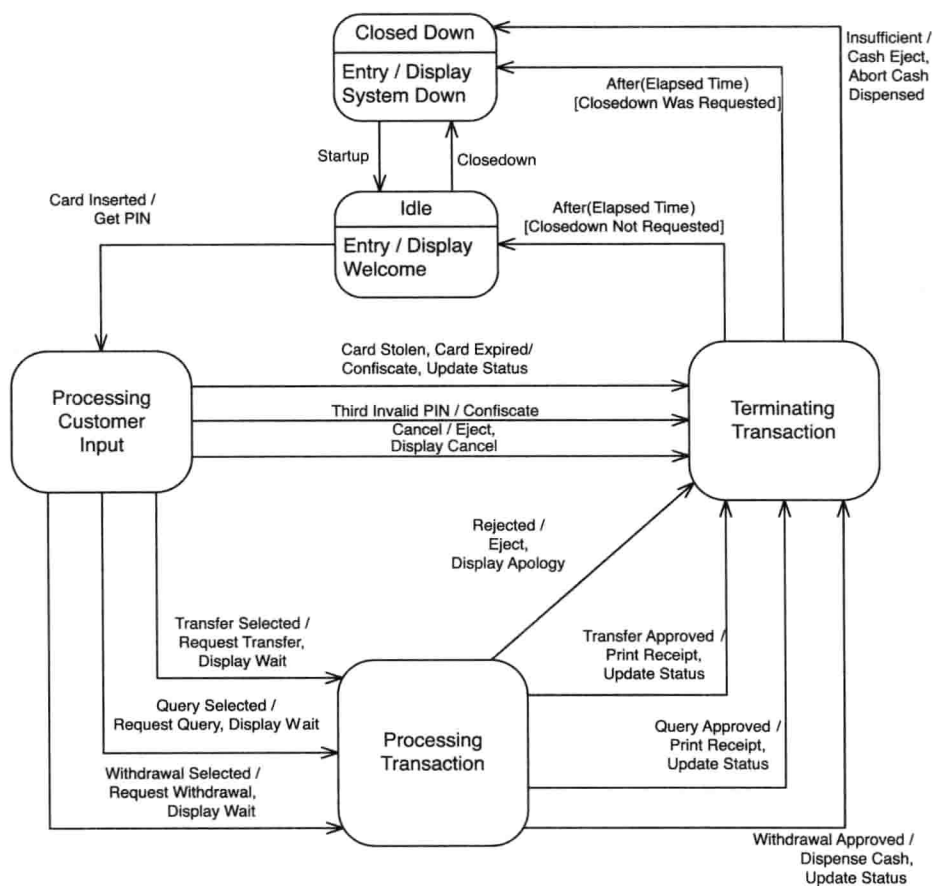
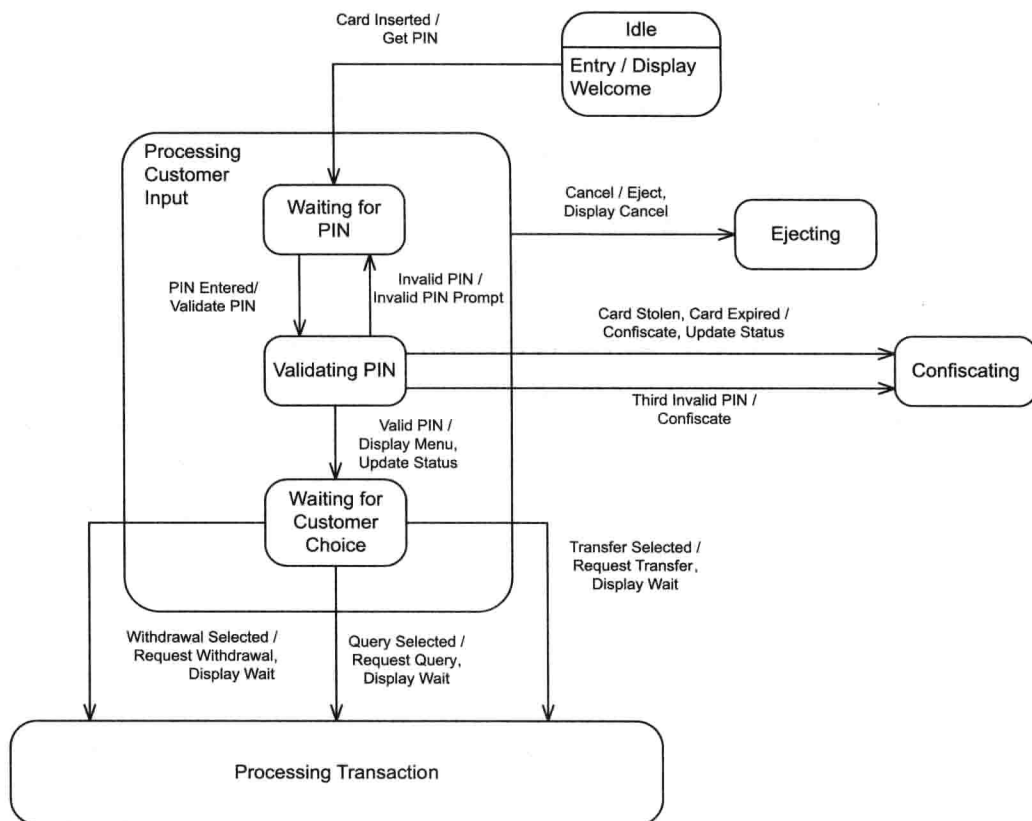


图 21-21 “ATM 控制”的顶层状态图

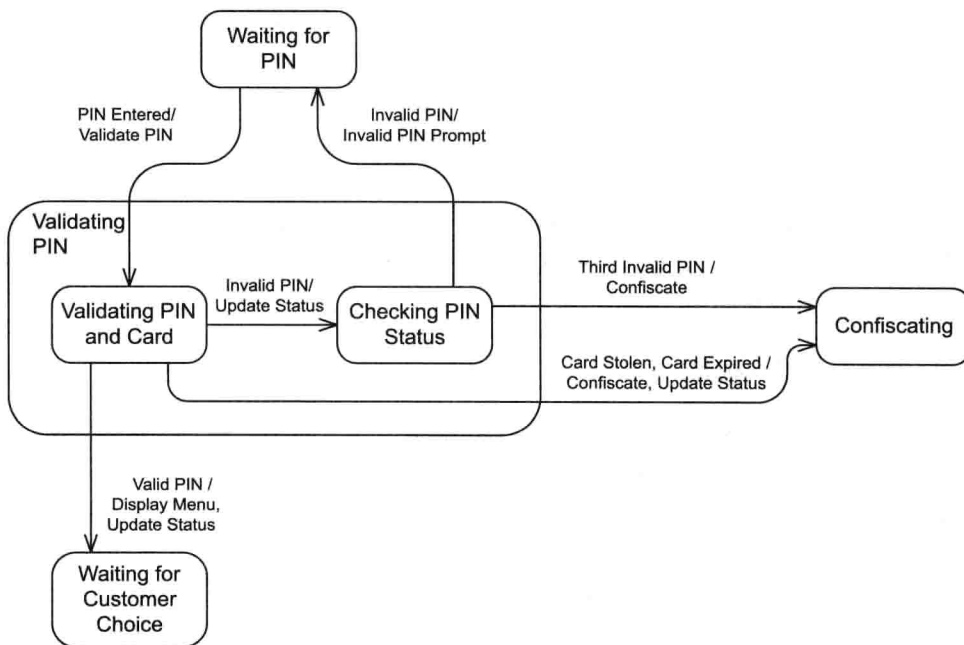
21.6.1 处理客户输入复合状态

“处理客户输入”复合状态（图 21-22）被分解为三个子状态：“等待 PIN 码输入”（Waiting for PIN）、“验证 PIN 码”（Validating PIN）以及“等待客户选择”（Waiting for Customer Choice）：

1) 等待 PIN 码输入：当客户把卡插入 ATM 机之后，ATM 机从“空闲”状态进入该子状态，从而触发“卡片插入”（Card Inserted）事件。在该子状态下，ATM 机等待客户输入 PIN 码。



a)



b)

图 21-22 “ATM 控制”的状态图：“处理客户输入”复合状态

2) 验证 PIN 码：当客户输入 PIN 码后 ATM 机进入该子状态。在该子状态下，“银行服务”验证 PIN 码。

3) 等待客户选择：在“有效 PIN 码”(Valid PIN)事件发生之后 ATM 机进入该子状态，意味着 PIN 码已经正确输入。在该子状态下，客户需要输入一个选择：取款、转账或查询。

该状态图是通过考虑客户参与者完成各用例(从“验证 PIN 码”用例开始)时 ATM 机的不同状态开发出来的。当一个客户插入 ATM 卡，“卡片插入”事件使得 ATM 机的状态变为“处理客户输入”复合状态的“等待 PIN 码输入”子状态(见图 21-22a)。在此期间，ATM 机一直等待客户输入 PIN 码。输出事件“获取 PIN 码”(Get PIN)使客户看到显示器上的提示。客户输入 PIN 码后，“PIN 码已输入”(PIN Entered)事件使 ATM 机的状态转移至“验证 PIN 码”子状态，此时“银行服务”会判断客户输入的 PIN 码是否与“银行系统”中储存的对应卡的 PIN 码相符。一共有三个来自“验证 PIN 码”状态的状态转移。如果所输入的 PIN 码正确，便可通过“有效 PIN 码”转移进入“等待客户选择”状态。如果 PIN 码不正确，将通过“无效 PIN 码”(Invalid PIN)转移重新进入“等待 PIN 码输入”状态让客户重新输入 PIN 码。如果客户错误输入 PIN 码三次，将通过“第三次错误输入 PIN 码”(Third Invalid)转移进入“结束交易”复合状态的“没收”(Confiscating)子状态。

“验证 PIN 码”子状态本身也是一个复合状态，由两个子状态组成：“验证 PIN 码和卡”(Validating PIN and Card)以及“检查 PIN 码状况”(Checking PIN Status)(见图 21-22b)。在前一个子状态下，系统会将(从卡上读出的)卡号和(客户输入的)PIN 码与存储于“卡账户”(Card Account)实体对象的卡号/PIN 码相比较，由此验证客户输入的卡号与 PIN 码是否正确。此外，验证卡号可以保证该卡未被挂失。如果验证通过，ATM 机转移到“等待客户选择”状态。如果卡已被挂失，那么 ATM 转移到“没收”状态。然而，如果是输入的 PIN 码不正确，那么需要额外检查此次输入是否是第三次错误的输入。最好把错误输入的计数存于客户端而非服务器，因为它只是一个 ATM 机的局部问题。因此，错误的 PIN 码输入次数被保存在“ATM 交易”对象中。每当服务器响应这是一个错误 PIN 码输入，该计数就被更新一次。如果计数小于三次，那么 ATM 转移回“等待 PIN 码输入”状态。如果计数表明这是“第三次错误输入 PIN 码”，那么 ATM 转移到“没收”状态。

在任意一个“处理客户输入”子状态中，客户都可以按 ATM 机上的“取消”按钮。此时，“取消”(Cancel)事件将 ATM 转移到“结束交易”复合状态的“弹出卡”子状态。由于“取消”事件可能发生在“处理客户输入”复合状态的任意子状态中，因此将“取消”转移画在复合状态上会显得更简洁。

21.6.2 处理交易复合状态

“处理交易”复合状态(图 21-23)也被分解为三个子状态，每个子状态针对一种交易：“处理取款”(Processing Withdrawal)、“处理转账”(Processing Transfer)和“处理查询”(Processing Query)。系统将根据客户的选择(例如取款)进入“处理交易”复合状态中相应的子状态(例如“处理取款”)以处理客户的请求。

在“等待客户选择”状态下，客户可以选择取款、查询或者转账并进入“处理交易”复合状态中相应的子状态，例如“处理取款”子状态(见图 21-23)。当取款交易完成后，如果客户有足够的资金，那么就会触发“允许取款”(Withdrawal Approved)事件，从而进入“结束交易”复合状态的“分发现金”子状态(见图 21-24)。相反，如果客户资金不足或者取款

金额超出每日取款上限，就会触发一个“拒绝”(Rejected)事件。

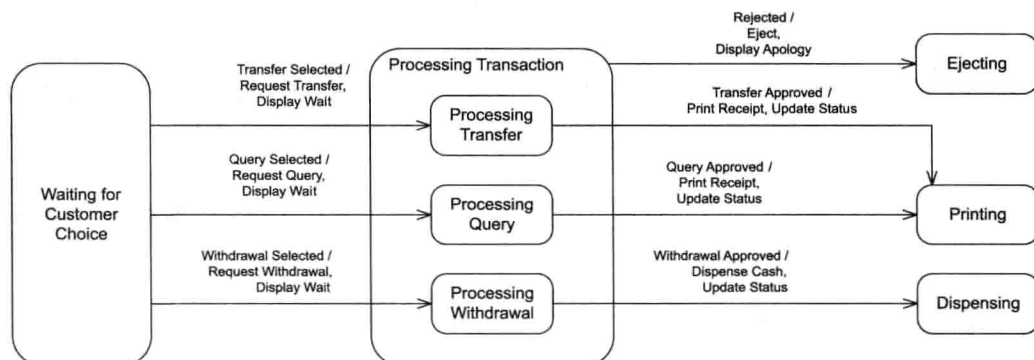


图 21-23 “ATM 控制”的状态图：“处理交易”复合状态

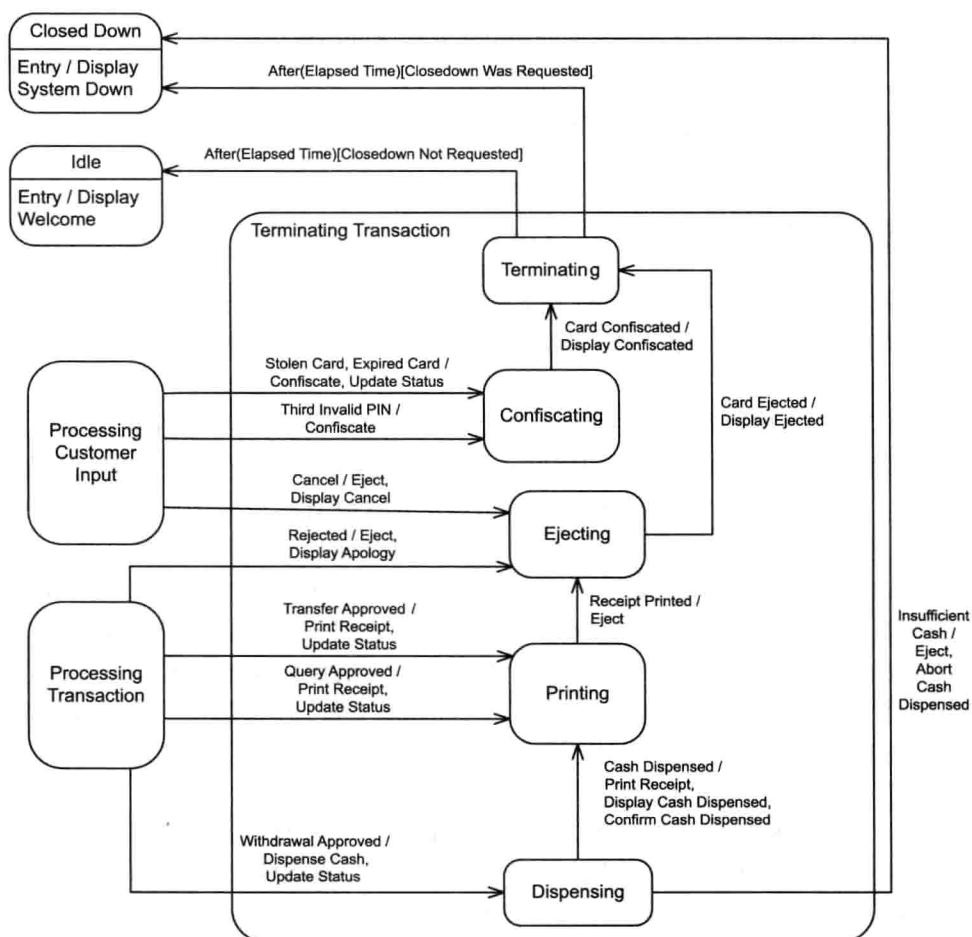


图 21-24 “ATM 控制”的状态图：“结束交易”复合状态

21.6.3 结束交易复合状态

“结束交易”复合状态（见图 21-24）的子状态包括“分发现金”(Dispensing)、“打印”

400 (Printing)、“弹出卡”(Ejecting)、“没收”(Confiscating)和“停止”(Terminating)。

与“分发现金”状态的转移相关的动作是“分发现金”(Dispense Cash)和“更新状态”(Update Status)。当“现金已分发”(Cash Dispensed)事件发生后,ATM机转移到“打印”状态下进行凭条打印。在该转移上执行“打印凭条”(Print Receipt)动作。凭条被打印完后,则进入“弹出卡”状态并执行“弹出卡”动作。当卡被弹出后(事件“卡弹出”(Card Ejected)),进入“停止”状态。

如AMT状态图所示,对于“查询”和“转账”交易,除了不会分发现金外,交易被允许后的状态变化序列都是相似的。

21.7 银行系统的设计

接下来,我们将“银行系统”的分析模型映射到一个设计模型,此过程可分为以下步骤:

- 1) 集成通信模型,开发出集成的通信图。
- 2) 将“银行系统”分成多个子系统并定义子系统之间的接口。
- 3) 对每个子系统,将其分解为多个并发任务。
- 4) 对每个子系统,设计信息隐藏类。
- 5) 开发更加详细的软件设计。

21.8 集成通信模型

由于“银行系统”是一个客户端/服务器系统(21.4节),因此此前我们已经确定将其分为客户端与服务子系统,如图21-8所示。通信图也相应地按照客户端和服务子系统进行了构造。

图21-11与图21-16描述了客户端“验证PIN码”用例和“取款”用例的通信图。同样,客户端“转账”用例、“查询账户”用例以及由操作员触发的用例也需要这样的通信图。如第13章所述,合并所有基于用例的通信图就可以得到“ATM客户端”子系统的集成通信图(图21-25)。为了完整性,集成的通信图必须覆盖每个用例的主序列和可替换序列。

一些对象参与了所有的客户端通信,如“ATM控制”;而另一些对象只参与了部分通信,如“吐钞器接口”(Cash Dispenser Interface)。集成通信图中有些消息是聚合消息,如“客户事件”(Customer Events)和“显示器提示”(Display Prompts)。集成的通信图还必须包括第13章提到的所有可替换序列。因此,“没收”(Confiscate)和“卡被没收”(Card Confiscated)消息来自于描述客户交易失败的可替换序列。与之相似,“显示器提示”聚合消息包含了处理错误PIN码输入的消息、客户账户金额不足的消息等。

401

现在考虑“银行服务子系统”。图21-14和21-19是服务器端“验证PIN码”用例和“取款”用例的通信图。此外,服务器端的“转账”用例和“查询账户”用例还需要额外的通信图。图21-26描述了“银行服务子系统”的集成通信图。每一种交易都对应一个封装交易业务逻辑的交易管理器对象,包括“PIN码验证交易管理器”(PIN Validation Transaction Manager)、“取款交易管理器”(Withdrawal Transaction Manager)、“查询交易管理器”(Query Transaction Manager)和“转账交易管理器”(Transfer Transaction Manager)对象。此外,在设计时还可以确定此处需要一个协调者对象,即“银行交易”(Bank Transaction)协调者。如第15章所述,这个协调者对象用来接收客户端请求并将它们委托给适当的交易管理器。

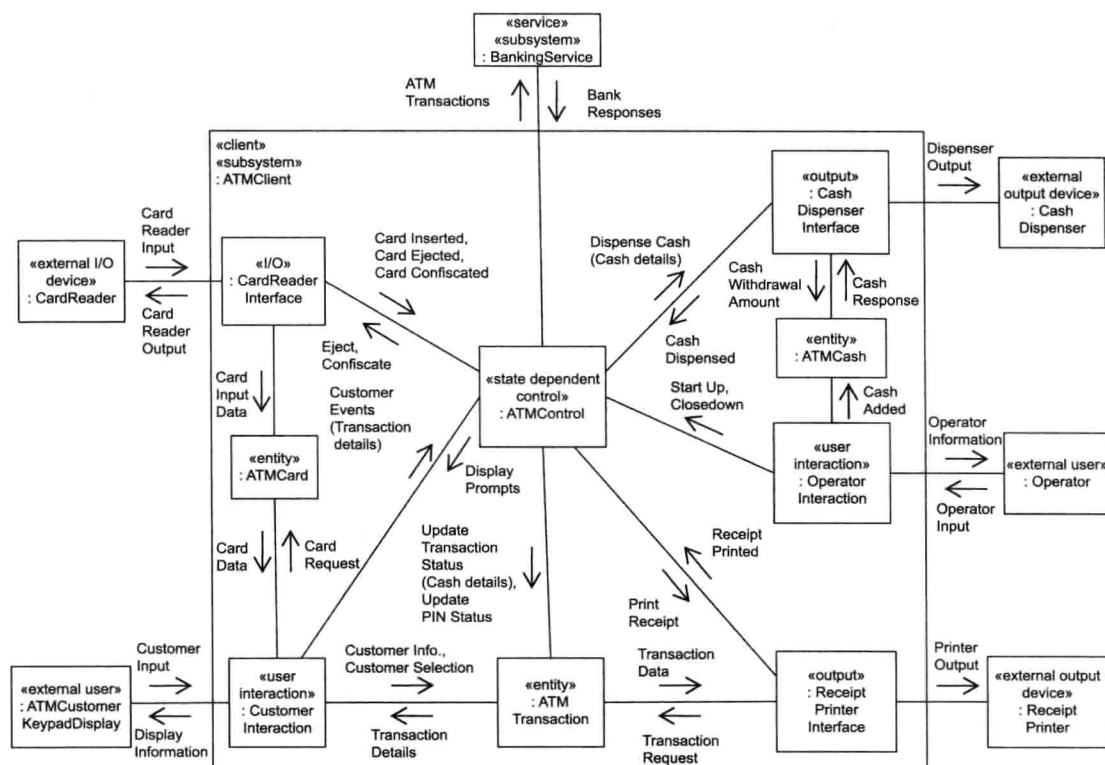


图 21-25 “ATM 客户端”子系统的集成通信图

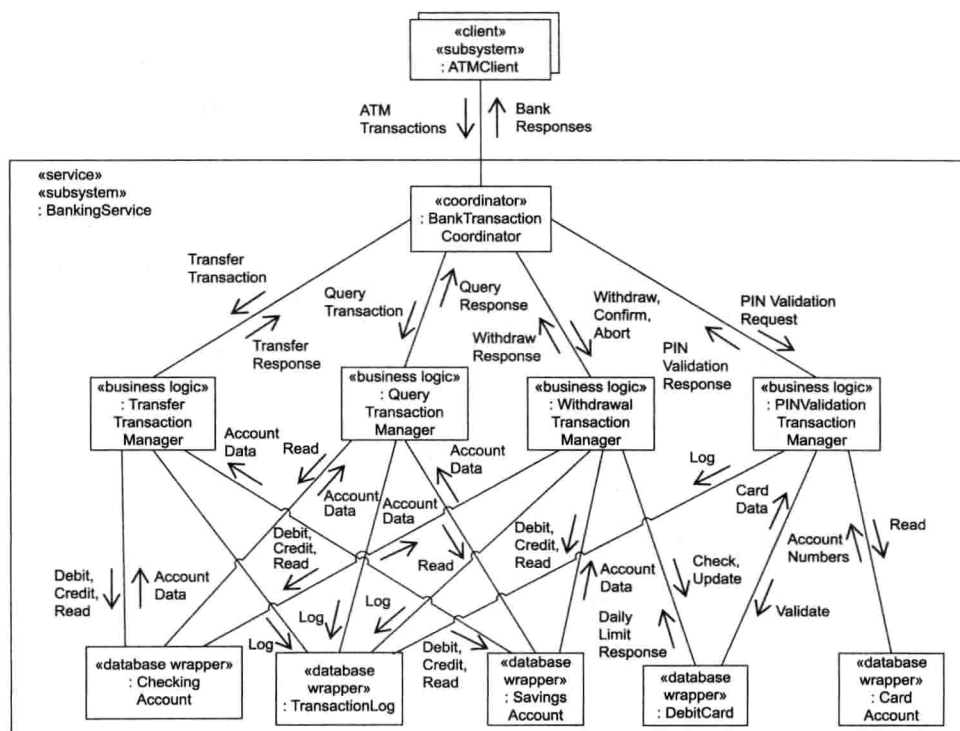


图 21-26 “银行服务子系统”的集成通信图

402
403

21.9 将系统划分为子系统

在“银行系统”的例子中，子系统划分步骤是显而易见的。“银行系统”是一个典型的基于多客户端 / 单服务体系结构模式的客户端 / 服务器体系结构。系统中一共包括两个子系统：拥有多个实例的“ATM 客户端子系统”和“银行服务子系统”，如此前的图 21-8 所示。这两个子系统也可以由图 21-27 所示的高层通信图来描述。

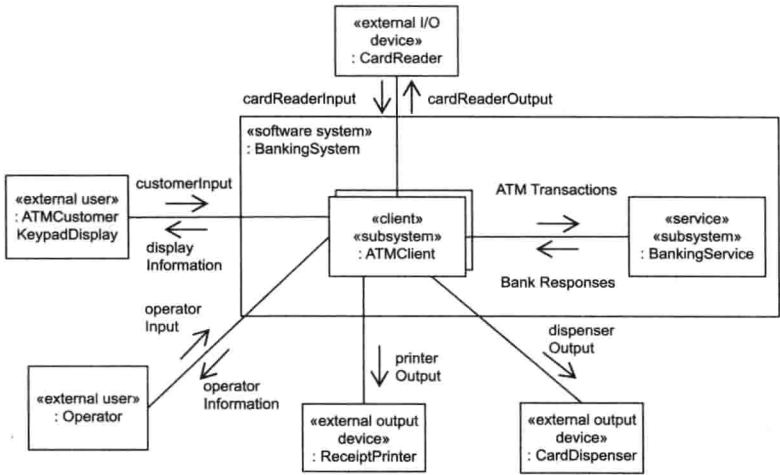


图 21-27 子系统设计：“银行系统”的高层通信图

图 21-27 是一个描述两个子系统间简单消息传递的分析层面的通信图。“ATM 客户端子系统”向“银行服务子系统”发送“ATM 交易”消息，并得到一个“银行响应”。“ATM 交易”消息是一个包含了“PIN 码验证”（PIN Validation）、“取款”（Withdraw）、“查询”（Query）、“转账”（Transfer）、“确认”（Confirm）以及“终止”（Abort）消息的聚合消息。“银行响应”是对这些消息的各种响应。

下一步考虑 ATM 应用的分布情况并定义分布式消息接口。由于这是一个客户端 / 服务器子系统，所以客户端子系统有多个实例而服务子系统有一个实例。每个子系统实例在自己的结点上运行。在设计模型中，每个子系统都是一个包含至少一个任务的并发系统。消息接口是带回复的同步消息通信。每个“ATM 客户端”向“银行服务”发送一个消息然后等待响应。因为“银行服务”会从多个 ATM 客户端收到消息，因此“银行服务”建立一个先进先出的消息队列来处理到达的消息。图 21-28 描述了设计模型通信图。

接下来考虑如何将每个子系统划分为多个并发任务。本章后续各节将首先考虑“ATM 客户端子系统”的设计，然后是“银行服务子系统”的设计。

21.10 ATM 客户端子系统的设计

要确定系统中的任务，就需要了解应用中各个对象之间的交互方式。分析模型的通信图最能阐述这一点，它描述了一个给定用例中对象间的消息传递顺序。对于“ATM 客户端子系统”，除了该子系统的集成通信图，还需要考虑“客户验证 PIN 码”（Client Validate PIN）用例和“客户取款”（Client Withdrawal Funds）用例的通信图。本节中描述的任务设计将产生图 21-29 中所描述的并发通信图。

404

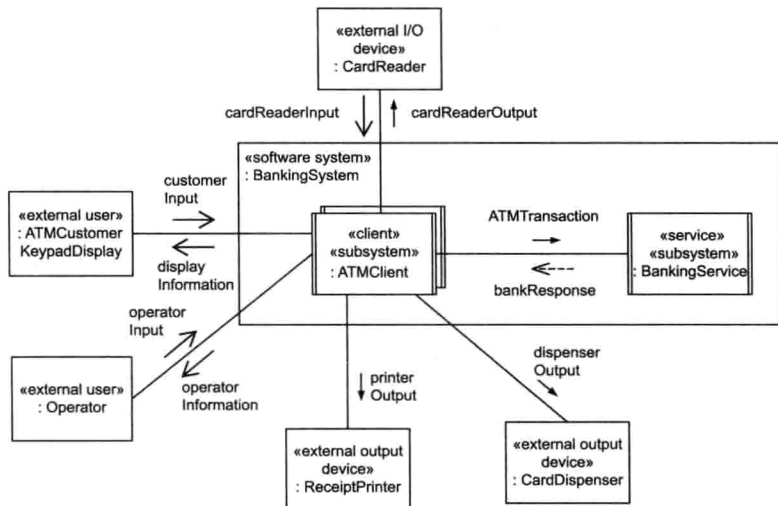


图 21-28 子系统接口：“银行系统”的高层并发通信图

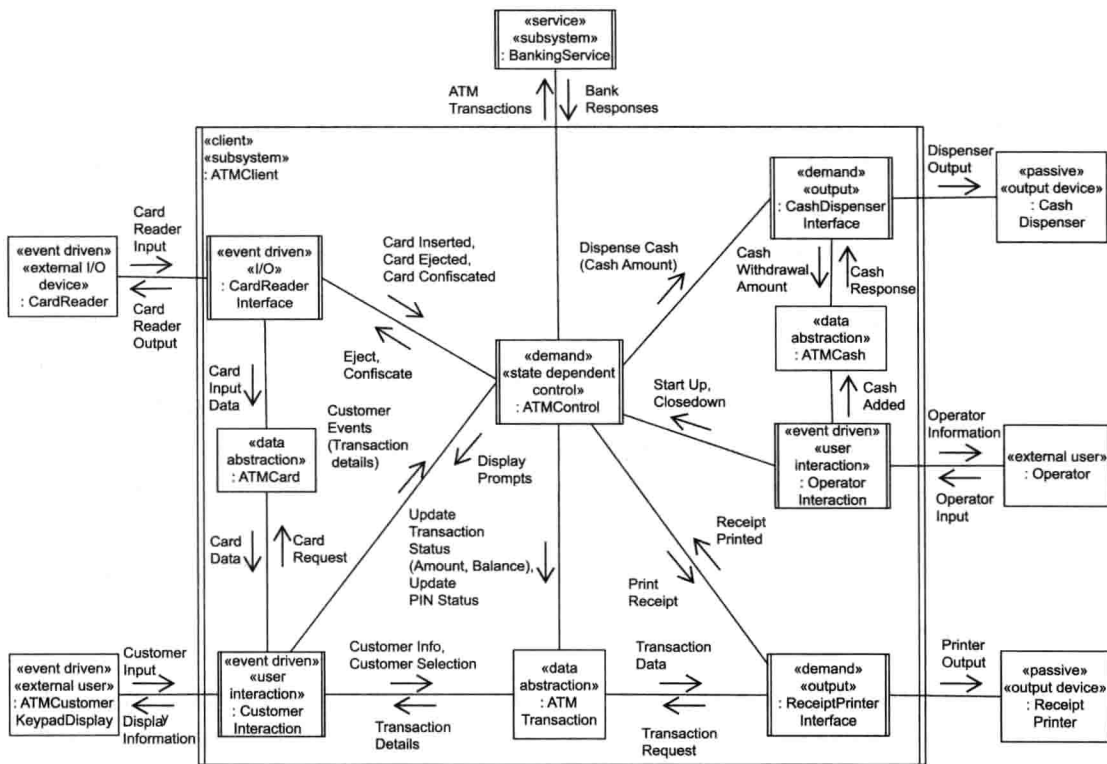


图 21-29 任务体系结构：初始的“ATM 客户端子系统”并发通信图

21.10.1 设计 ATM 子系统并发任务体系结构

考虑支持“验证 PIN 码”用例的通信图（见图 21-11）。参与交互的第一个对象是“读卡器接口”（Card Reader Interface）对象，它是一个与真实读卡器交互的输入/输出对象。“读卡器”外部输入/输出设备是一个事件驱动的输入/输出设备，其特点是在得到输入时它会产生

一个中断。“读卡器接口”对象被构造为一个事件驱动输入/输出任务，如图 21-29 所示。一开始，该任务处于休眠状态。它由中断触发，然后读取读卡器输入并将其转为内部格式。接着它把卡的内容写到“ATM 卡”实体对象上。由于“ATM 卡”是一个被动对象，所以不需要独立的控制线程。该对象进一步被归类为一个数据抽象对象。

“读卡器接口”任务然后向“ATM 控制”发送一个“卡片插入”(Card Inserted)消息，“ATM 控制”是一个状态相关的控制对象，它执行“ATM 控制”状态图。“ATM 控制”对象被构建为一个状态相关的按需驱动的控制任务，这是因为它需要独立的控制线程来处理不同来源的消息。一开始，它一直处于空闲状态，当有包含控制请求的消息到达时它被激活。收到“卡片插入”消息后，“ATM 控制”执行状态图并转移到“等待 PIN 码输入”子状态(见图 21-21 和图 21-22)。与该状态转移关联的活动向“客户交互”发送一个“获取 PIN 码”消息，“客户交互”是一个与用户交互的对象，它在显示器上输出消息并从键盘上接收输入。“客户交互”被构建为一个事件驱动的用户交互任务，它拥有自己独立的控制线程。它提示客户输入 PIN 码、接收 PIN 码、从“ATM 卡”读取卡信息然后将卡和 PIN 码信息写到“ATM 交易”对象上，“ATM 交易”是一个被动的数据抽象对象。由于“ATM 卡”对象和“ATM 交易”数据抽象对象会被许多任务访问，所以它们被放在任何一个任务之外。

接下来，考虑支持“取款”用例的通信图，其中包含了很多“验证 PIN 码”通信图中的对象。不同于“验证 PIN 码”通信图的对象有“凭条打印机接口”(Receipt Printer Interface)、“吐钞器接口”(Cash Dispenser Interface)以及“ATM 现金”(ATM Cash)。

外部的“吐钞器”是一个被动的输出设备，因此它不需要一个事件驱动的输出任务。故而将“吐钞器接口”对象作为一个按需驱动(demand driven)的输出任务，它会被从“ATM 控制”发送来的请求消息激活。与之相似，“凭条打印机接口”对象也被构建为一个按需驱动的输出任务，它会被从“ATM 控制”任务发送来的请求消息激活。

ATM“操作员交互”用户交互对象参与了三个由操作员引发的用例，它也被映射为一个事件驱动的用户交互任务(见图 21-29)。“ATM 现金”实体对象是一个被动的数据抽象对象，因此不需要独立的控制线程，它会被“吐钞器接口”和 ATM“操作员交互”任务访问。

综上所述，一共有三个事件驱动的输入/输出任务(“读卡器接口”)、一个按需驱动的状态相关控制任务(“ATM 控制”)、两个按需驱动的输出任务(“吐钞器接口”和“凭条打印机接口”)、两个事件驱动的用户交互任务(“客户交互”和 ATM“操作员交互”)。此外，还有三个被动实体对象(“ATM 卡”、“ATM 交易”、“ATM 现金”)，它们都被归类为数据抽象对象。

21.10.2 定义 ATM 子系统任务接口

为了确定任务接口，有必要分析对象(主动或被动)间的交互方式。首先，考虑由被动数据抽象对象确定的任务交互。在每种情况下，任务会调用被动对象提供的操作。这必然是一个同步调用，因为操作处于该任务的控制线程之中。同样地，数据抽象对象的所有操作都会被同步调用。由于每个被动对象都会被多个任务调用，因此有必要同步对这些数据的访问。在下一节中详述这些被动对象的操作。

接下来考虑任务间的消息交互。先考虑“读卡器接口”任务和“ATM 控制”任务之间的接口。我们希望“读卡器接口”向“ATM 控制”发送一个消息之后不需要等待消息被接收。为此，需要一个异步消息接口，如图 21-30 所示。这意味着还需要一个反向的消息接口，因为“ATM 控制”需向“读卡器接口”任务发送“弹出”(Eject)和“没收”(Confiscate)消

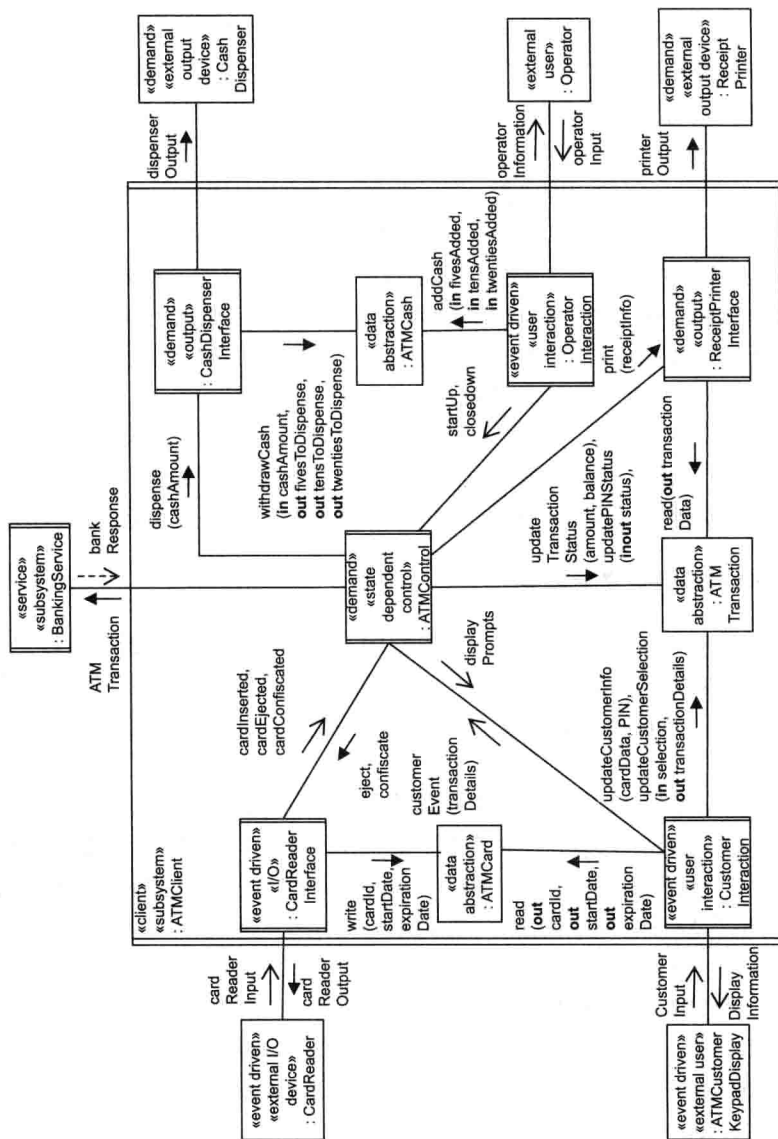


图21-30 任务体系结构：修正后的“ATM客户端子系统”的并发通信图

407
408

息。我们将此处设计为一个不带回复的同步消息接口，在向“ATM 控制”发送完消息之后，“读卡器接口”将一直等待“弹出”或者“没收”返回消息。这意味着“ATM 控制”可以发送一个同步消息，而不必等待“读卡器接口”接收此消息。“读卡器接口”任务的响应是异步的，这就提供了“读卡器接口”和“ATM 控制”任务之间最大的灵活性。

考虑“客户交互”和“ATM 控制”之间的接口。应该是异步还是同步呢？首先，考虑一个带响应的同步场景。“客户交互”向“ATM 控制”发送一个“取款”请求，“ATM 控制”然后再向“银行服务”发送交易。在接收到服务器的响应后，“ATM 控制”向“客户交互”发送一个显示器提示。这种情况下，“客户交互”不可能与客户进行交互，因为该对象处于挂起状态并等待“ATM 控制”的响应。这从客户的角度而言是不合理的。然后，考虑异步接口的方案，如图 21-30 所示。按照此方案，“客户交互”向“ATM 控制”发送一个“取款”请求而无需等待响应。这样，在得到服务器的响应之前，“客户交互”可以对用户的“取消”请求做出响应。“客户交互”从“ATM 控制”接收作为一个独立的异步消息接口的响应。“客户交互”被设计为既可以从客户也可以从“ATM 控制”收到消息的任务，哪个消息先来就先处理哪一个。

ATM“操作员交互”任务的接口也是异步的。操作员参与者的请求独立于客户请求，由此，从客户和从操作员到“ATM 控制”的消息可以以任意顺序到达。所以，“ATM 控制”用一个先进先出的消息队列来处理到达的消息。

两个输出任务（“吐钞器接口”和“凭条打印机接口”）会被从“ATM 控制”到达的请求消息激活。在这种情况下，输出任务在消息到达之前处于空闲状态，因此可以采用同步接口，因为它不会阻碍“ATM 控制”的运行。在图 21-30 中，我们更新了并发交互图来显示任务接口。

21.10.3 设计 ATM 客户端信息隐藏类

“银行系统”中的对象和类最初是在分析模型中确定的。在设计过程中可以进一步对被动类进行划分，例如实体类可以被进一步划分为数据抽象类或数据库包装器类。在设计类的过程中还要设计类的接口，如第 14 章所述。为了确定类接口，有必要考虑通信图中对象相互之间是如何交互的。

首先，考虑“ATM 客户端子系统”中实体类的设计。由于“ATM 客户端子系统”中没有数据库，所有的实体类都是封装自己的数据，因此可以被进一步归类为数据抽象类。“ATM 客户端子系统”有三个数据抽象类：“ATM 卡”、“ATM 交易”和“ATM 现金”。数据抽象类的属性在实体类的概念静态建模中确定，如第 21.3 节所述。通过分析这些类在交互图中的使用方式可以确定这些类的操作。

在第 14 章中已经描述了“ATM 现金”类和“ATM 卡”类的设计。对于“ATM 交易”类，它的属性由静态模型确定，但它的操作是根据其他对象访问该类的方式确定的，访问方式已在通信图中给出。该类的操作有“更新客户信息”（update Customer Information）、“更新客户选择”（update Customer Selection）、“更新 PIN 码状态”（update PIN Status）、“更新交易状态”（update Transaction Status）以及“读取”（read）。前两个操作由“客户交互”任务调用。接下来两个操作由“ATM 控制”任务调用。“凭条打印机接口”任务在打印凭条之前调用“读取”操作。

还有一个状态机类，即“ATM 状态机”（ATM State Machine）类，它处于“ATM 控制”任务之内并封装了 ATM 状态图，该类被实现为一个状态转移表。它的操作包括“处理事件”（process Event）和获取“当前状态”（current State），这些都是状态机类的标准操作。

图 21-31 详细描述了类的设计，包括了类的属性和操作。

409

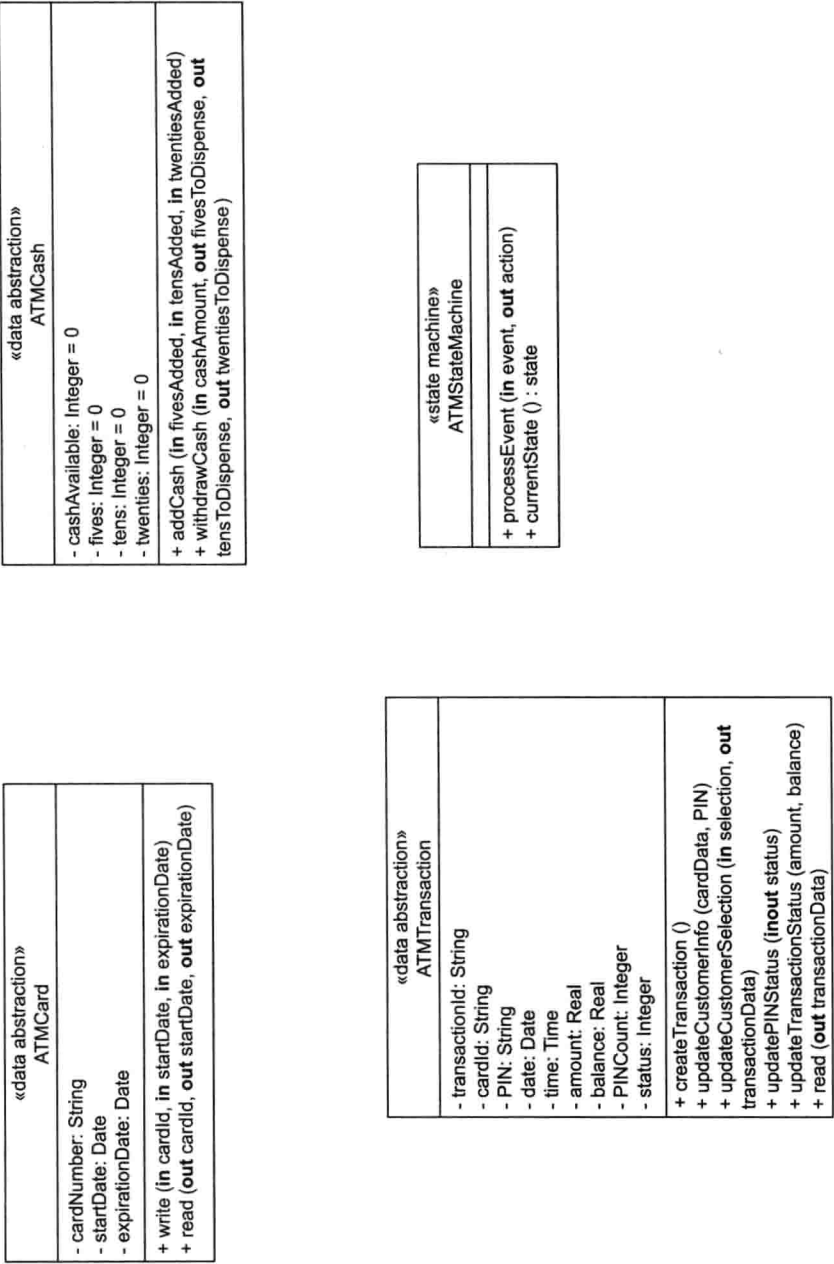


图21-31 “ATM客户端”信息隐藏类

21.11 银行服务子系统的设计

由于银行服务器拥有“银行系统”的中央数据库，因此我们通过考虑与静态模型相关的重要设计决策来开始“银行服务子系统”的设计。实体类的概念静态模型（见图 21-4—图 21-7）包含了几个存在于银行服务器的实体类。我们做出如下设计决策：在服务器的实体类（同样也是原先在问题域的静态模型中所描述的实体类）需要被存储为关系数据库中的关系表。故而在设计时我们确定服务器的实体类不封装任何数据，而是封装针对关系数据库的接口，它们本质上是数据库包装器类。在下一节中详述数据库包装器类的设计以及实体类模型到关系数据库的映射。

21.11.1 设计银行服务子系统并发任务体系结构

现在考虑“银行服务子系统”的设计。我们决定使用一个顺序性服务。只要服务器的处理能力足够大，这样的设计就没有问题。在一个顺序性服务中，服务被设计为一个任务，即设计为只有一个控制线程的程序。这种顺序性服务在一个先进先出的消息队列上处理每个交易，前一个交易处理完后下一个交易才能开始。

我们将“银行服务子系统”设计为一个顺序性服务任务，并按需激活运行。该任务中包括协调者对象（“银行交易协调者”）、业务逻辑对象（“PIN 码验证交易管理器”、“取款交易管理器”、“查询交易管理器”以及“转账交易管理器”），以及实体对象（可以被进一步归类为数据库包装器类）。服务子系统的初始任务设计只包含一个任务，如图 21-32 所示。

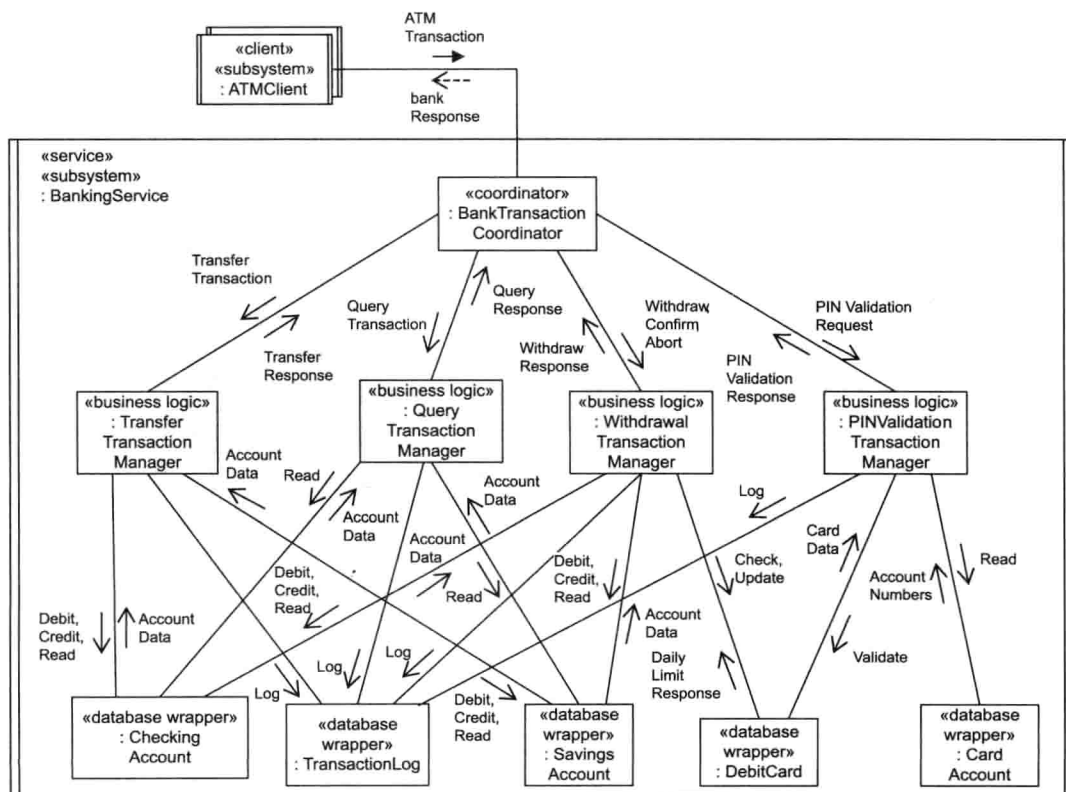


图 21-32 “银行服务子系统”的初始并发通信图

“银行交易协调者”任务接收到达的交易消息并做出响应。它委托交易管理器处理交易，而交易管理器随后再访问数据库包装器类。“银行服务子系统”内部的所有交互都是同步的，这与下文所述的操作调用相对应。

21.11.2 设计银行服务信息隐藏类

第 15 章阐述了数据库包装器类的设计以及如何从分析模型的实体类映射到设计模型的数据库包装器类和关系数据库的关系表（扁平文件）。在“银行服务”中，数据库包装器类包括“账户”（Account）、“支票账户”（Checking Account）、“储蓄账户”（Savings Account）、“借记卡”（Debit Card）、“卡账户”（Card Account）以及“交易日志”（Transaction Log），如图 21-33 所示。每个类都封装了表示数据库中关系的接口。从数据库的角度来看，一个关系数据库仅包含扁平文件，并不支持类层次结构，因此“账户”的泛化/特化层次结构也相应被扁平化，故而“账户”父类的属性在“支票账户”与“储蓄账户”中都存在（如第 15 章所述）。然而，对于“银行服务”中数据库包装器类的设计，“账户”的泛化/特化层次结构需要保留，从而使“支票账户”和“储蓄账户”数据库包装器类继承抽象的“账户”父类的泛化操作。

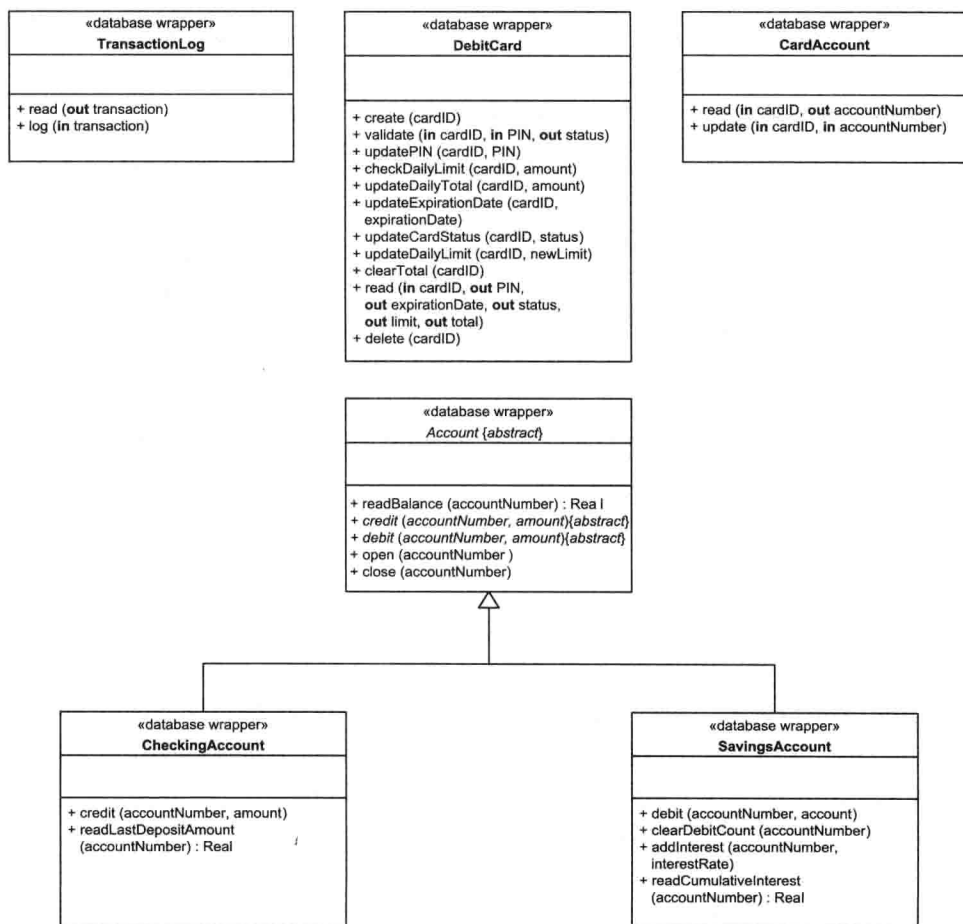


图 21-33 银行服务的数据库包装器类

413

还需要设计四个业务逻辑类的接口，它们是“PIN 码验证交易管理器”、“取款交易管理器”、“查询交易管理器”以及“转账交易管理器”，如图 21-34 所示。每个交易管理器都处理一种原子交易。例如，“取款交易管理器”提供了一个“取款”操作，在处理客户取款请求时会调用该操作。当一个“ATM 客户端”对象确认现金已分发给客户时会调用“确认”操作。当一个“ATM 客户端”对象终止交易时会调用“终止”操作，这种情况可能是提款机分发现金失败，也可能是客户取消了交易。

21.11.3 设计银行服务接口

“银行服务”是只有一个控制线程的顺序性服务子系统。在这一阶段，尤其需要考虑“银行服务”任务的设计。该任务是一个由被动对象组成的复合任务。“银行交易协调者”对象接收到达的交易并将它们委托给业务逻辑对象，即“PIN 码验证交易管理器”、“取款交易管理器”、“查询交易管理器”以及“转账交易管理器”。

414

“银行交易协调者”对象从“ATM 客户端”以先进先出的方式处理消息队列。对每个消息，该对象确定交易的类型，然后委托相应的交易管理器处理交易。当交易处理结束后，交易管理器会向“银行交易协调者”发送响应消息，进而再由“银行交易协调者”向“ATM 客户端”发送响应消息。以上步骤结束后，“银行交易协调者”再处理下一个交易消息。

图 21-32 显示了“银行服务子系统”的初始设计。在“银行服务”的初始并发通信图中，所有的接口都是简单消息。图 21-35 显示了“银行服务子系统”并发通信图的最终版本。多个 ATM 客户端的实例与“银行服务”之间进行带回复的同步通信。“银行服务”内部的所有交互都只存在于被动对象之间；因此，所有的内部接口都用操作调用来定义（用同步消息表示法来表示）。

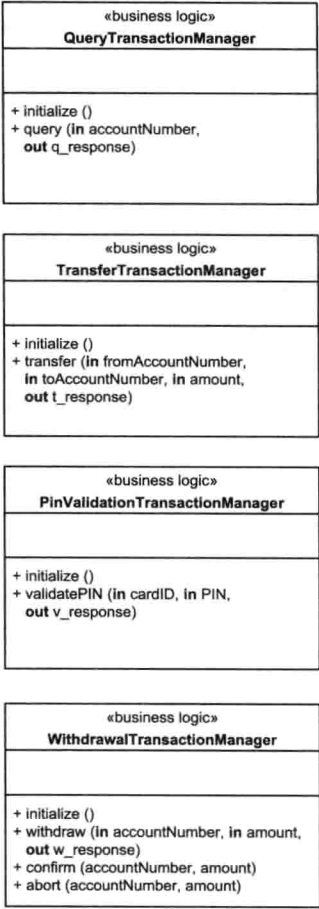


图 21-34 “银行服务”的业务逻辑类

21.12 关系数据库设计

本节从概念实体类模型（如 21.3.3 节所述以及图 21-4 ~ 图 21-7 所描述）开始来介绍银行关系数据库的逻辑设计。类图中描述的所有实体类（图 21-4）都存在于银行服务器之中。由于这些实体类的数据要被持久化，因此它们需要被存储在数据库中。如 21.12 节所述，实体类被设计为数据库包装器类，而实体类的内容（由实体类的属性所定义）需要保存在数据库中的关系表中。在下面的描述中，我们用下划线表示主键，用斜体表示外键。

根据静态模型来设计关系数据库的指导原则已在 15.5 节中介绍。现在考虑图 21-4 中的实体类。“银行”（Bank）、“ATM 信息”（ATM Info）、“客户”（Customer）以及“借记卡”（Debit Card）这些实体类都会被映射为一个关系表。对于每张表，能用来唯一确定表中某一行的属性就作为主键，如“客户”表的主键 customerId（客户号）。而外键则被用来实现表与表之间的导航。

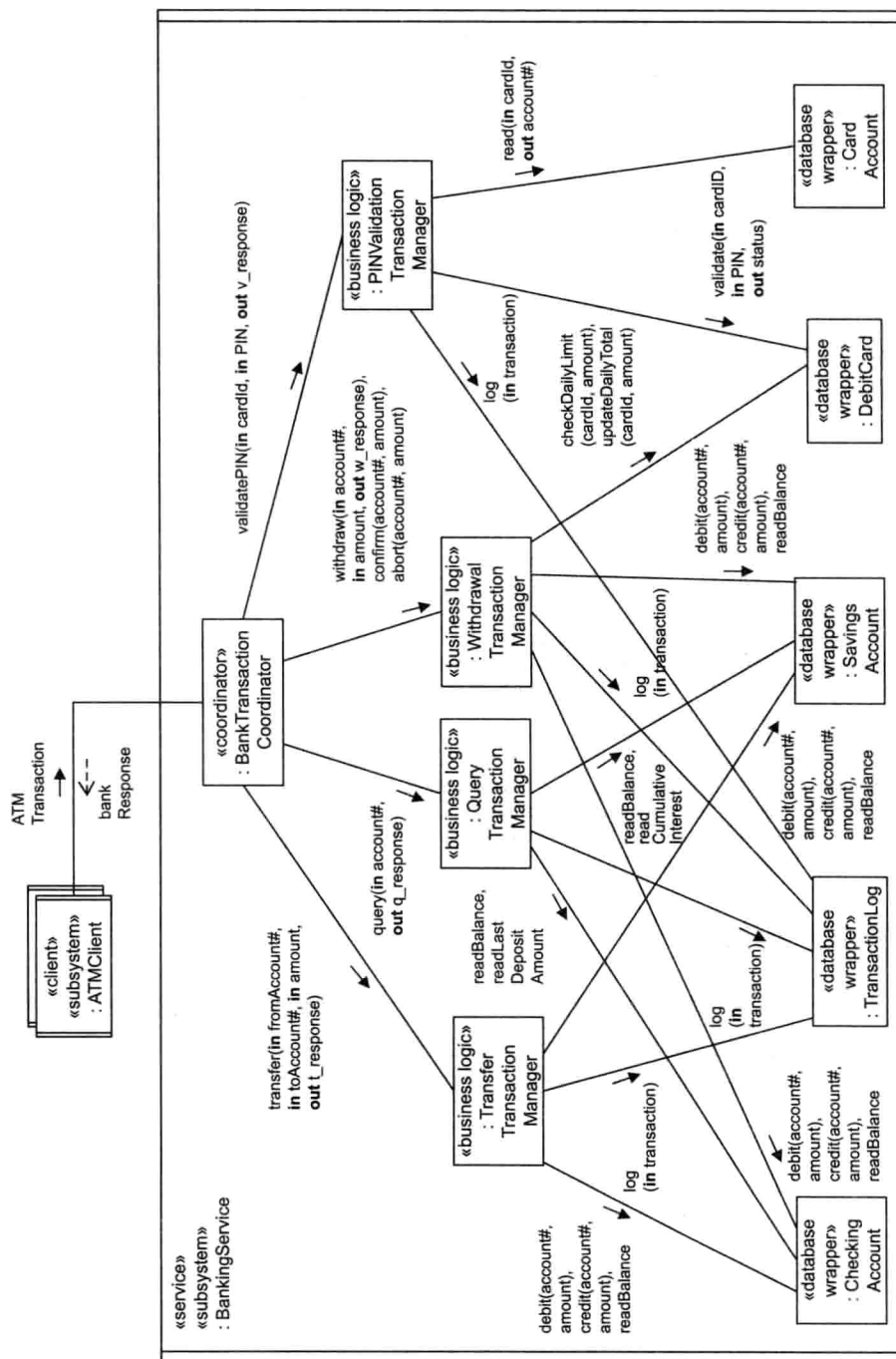


图21-35 修正后的“银行服务子系统”并发通信图

对于“账户”泛化/特化层次结构,我们决定通过将“账户”(Account)父类的属性放在“支票账户”(Checking Account)与“储蓄账户”(Savings Account)表中来实现层次结构的扁平化。虽然账户类型(储蓄账户或者支票账户)是“账户”类的一个属性,但是我们假定知道“账户号”后可以确定账户类型,因此“支票账户”和“储蓄账户”表的主键是 accountNumber (账户号)。与之相关联的“卡账户”类(在图 21-4 中定义)被设计为一张关联表,用来表示“卡”和“账户”之间的多对多关系。“客户账户”也被设计为一张关联表,用来表示“客户”和“账户”之间的多对多关系。即使在静态模型中没有“客户账户”这样一个关联类(因为 ATM 交易中并不需要它),在关系数据库中也需要这样一张表。

对于“ATM 交易”泛化/特化层次结构,我们对其进行扁平化处理,但只提供交易子类的关系表。一个 ATM 交易的主键是交易号,它由多个字段的复合主键组成,包括:bankId (银行号)、ATMId (ATM 号)、date (日期)、time (时间)。bankId 和 ATMId 也是外键,因为它们可以导航到“银行”表和“ATM 机信息”表。“ATM 机信息”有一个复合主键,由 bankId 和 ATMId 组成,其中 bankId 也是一个外键。日期和时间属性可以提供一个时间戳来唯一确定一笔交易。

```

Bank (bankName, bankAddress, bankId)
Customer (customerName, customer Id, customerAddress)
Debit Card (cardId, PIN, startDate, expirationDate, status, limit, total,
customerId)
Checking Account (accountNumber, accountType, balance,
lastDepositAmount)
Savings Account (accountNumber, accountType, balance, interest)
Card Account (cardId, accountNumber)
Customer Account (customerId, accountNumber)
ATM Info (bankId, ATMId, ATMLocation, ATMAddress)
Withdrawal Transaction (bankId, ATMId, date, time, transactionType, cardId,
PIN, accountNumber, amount, balance)
Query Transaction (bankId, ATMId, date, time, transactionType, cardId, PIN,
accountNumber, balance)
Transfer Transaction (bankId, ATMId, date, time, transactionType, cardId, PIN,
fromAccountNumber, toAccountNumber, amount)
PIN Validation Transaction (bankId, ATMId, date, time, transactionType,
cardId, PIN, startDate, expirationDate)

```

21.13 银行系统的部署

由于这是一个客户端/服务器系统,所以客户端子系统有多个实例而服务子系统只有一个实例。如图 21-36 的部署图所示,每个子系统实例在自己的结点上运行。因此,每个 ATM 客户端实例在 ATM 结点上运行,“银行服务”实例在服务器结点上运行。

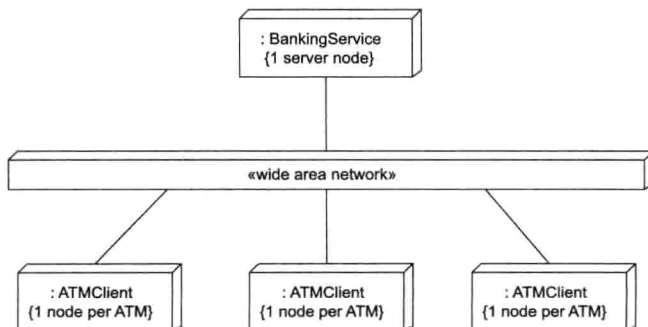


图 21-36 “银行系统”的部署图

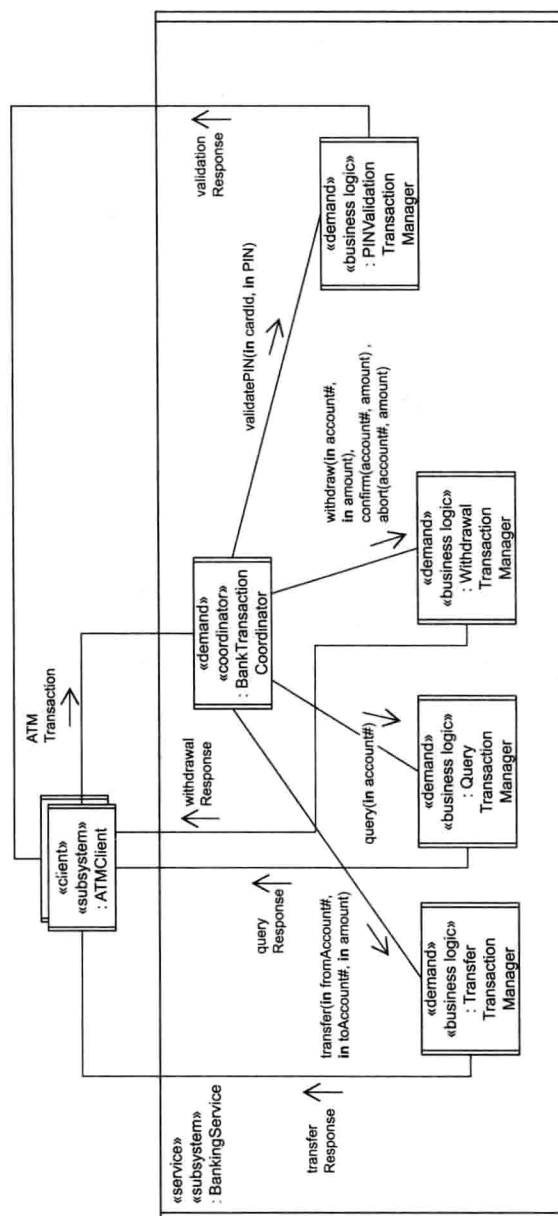


图21-37 “银行服务”的另一种设计选择：“银行系统”的并发服务设计

21.14 其他设计考虑

另一个设计决策是将“银行服务”设计为一个并发服务，其中“银行交易协调者”和每个业务逻辑对象都设计为独立的按需驱动、按需激活的任务。根据这种并发服务设计，“银行交易协调者”委托业务逻辑对象处理一个交易请求后马上处理下一个请求，由此服务器可以同时处理多个交易请求。当顺序性服务设计不足以应对交易的负载量时应该采取这种办法。关于更多的并发服务设计的信息请参阅第 15 章。

21.15 详细设计

我们用任务事件顺序逻辑（task event sequencing logic）来描述“银行系统”的详细设计。在第 18 章中已给出了“ATM 客户端子系统”中“读卡器接口”任务和“ATM 控制”任务的行为规约以及“银行服务子系统”中“银行服务”任务的行为规约。本节描述这些任务的事件顺序逻辑。

21.15.1 读卡器接口任务的事件顺序逻辑示例

“读卡器接口”任务（见图 21-30）会被一个外部读卡器事件唤醒，接着读取“ATM 卡”（ATM Card）输入、将卡内容写入“ATM 卡”对象、向“ATM 控制（ATM Control）”发送一个“卡片插入”（cardInserted）消息，然后等待消息。如果“ATM 控制”发送回来的消息是“弹出卡”，那么就弹出 ATM 卡；如果发送回来的消息是“没收”，那么就没收卡。被动的数据抽象对象“ATM 卡”处于任务之外，用于保存卡的内容。

“银行系统”中的所有消息通信都是通过调用操作系统来完成的。因此，“读卡器接口”任务（生产者）和“ATM 控制”（消费者）之间的消息队列 ATMControlMessageQ 由操作系统提供，而“ATM 控制”和“读卡器接口”任务之间的同步通信也是如此（见图 21-32）。一个名为 cardReaderMessageBuffer 的消息缓存接收来自“ATM 控制”的同步消息。

```
Initialize card reader;
loop
-- 等待来自读卡器的外部中断
wait (cardReaderEvent);
Read card data held on card's magnetic strip;
if card is recognized
then -- 将卡数据写到 ATM 卡对象中
    ATMCard.write (cardID, startDate, expirationDate);
    -- 向 ATM 控制对象发送卡已插入的消息
    send (ATMControlMessageQ, cardInserted);
    -- 等待来自 ATM 控制的消息
    receive (cardReaderMessageBuffer, message);
    if message = eject
    then
        Eject card;
        -- 向 ATM 控制对象发送卡已弹出的消息
        send (ATMControlMessageQ, cardEjected);
    elseif message = confiscate
    then
        Confiscate card;
        -- 向 ATM 控制对象发送卡已没收的消息
        send (ATMControlMessageQ, cardConfiscated);
    else error condition;
```

```

end if;
else --未能识别卡，因此弹出
    Eject card;
end if;
end loop;

```

21.15.2 ATM 控制任务的事件顺序逻辑示例

“ATM 控制”任务处于“ATM 客户端”子系统的核心部分（见图 21-30），它与其他任务保持交互。“ATM 控制”有一个名为 `ATMControlMessageQ` 的输入消息队列，从那里接收来自三个生产者（“读卡器接口”、“客户交互”、ATM “操作员交互”）的消息。“ATM 控制”向一些任务发送消息。它向“读卡器接口”发送不带回复的同步消息，向“吐钞器接口”和“凭条打印机接口”任务发送带回复的同步消息，向“客户交互”任务的 `promptMessageQueue` 消息队列发送异步消息，向“银行服务”发送带回复的同步消息。

由于“ATM 控制”任务是状态相关的，因此它并不直接处理到达的事件而是执行状态图中定义的状态相关的动作。“ATM 状态机”对象中封装了状态图的实现，该对象嵌套在“ATM 控制”中。当新的事件发生时，“处理事件”操作返回待执行的活动。在“ATM 控制”输入消息队列中可接收大多数事件，但有三个例外。由于与“银行服务”的通信是同步的，因此来自“银行服务”的响应是作为 `send` 消息的输出参数收到的。由于与“吐钞器接口”和“凭条打印机接口”任务的通信也是同步的，因此“分发现金”和“打印凭条”动作是带回复的同步消息，返回相应的分发动作和打印动作是否成功的布尔值。

当同步消息的响应产生了新的内部事件时，我们会将 `newEvent` 变量设为该事件的值并将布尔变量 `outstandingEvent` 设为 `True`。该内部事件的一个例子是 `withdrawalResponse`（可能同时出现多个同步响应，在下一节“银行服务”的事件顺序逻辑中会阐述这种情况）和 `cashDispensed`。下面是描述了大部分由“ATM 控制”所执行的动作的事件顺序逻辑。执行每个动作后，下一步会跳转到伪代码 `case` 代码块的底部（为了描述简洁，伪代码中并未明确写出这种跳转）。“ATM 控制”任务的伪代码如下：

[420]

```

loop
    -- 来自所有发送者的消息都通过消息队列接收
    Receive (ATMControlMessageQ, message);
    -- 抽取事件名称和所有消息参数
    -- 针对收到的事件，查询状态转移表
    -- 根据需要改变状态，返回将要执行的动作
    newEvent = message.event
    outstandingEvent = true;
while outstandingEvent do
    ATMStateMachine.processEvent (in newEvent, out action);
    outstandingEvent = false;
    -- 执行ATM控制状态图中定义的动作
case action of
    Get PIN: -- 提示PIN码
        send (promptMessageQueue, displayPINPrompt);
    Validate PIN: -- 通过银行服务验证客户所输入的PIN码
        send (Banking Service, in validatePIN, out
            validationResponse);
        newEvent = validationResponse; outstandingEvent = true;
    Display Menu: -- 向客户显示选择菜单
        send (promptMessageQueue, displayMenu);
        ATMTransaction.updatePINStatus (valid);

```

```

Invalid PIN Action: -- 显示非法PIN码的提示
    send (promptMessageQueue, displayInvalidPINPrompt);
    ATMTransaction.updatePINStatus (invalid);
Request Withdrawal: -- 向银行服务发送取款请求
    send (promptMessageQueue, displayWait);
    send (Banking Service, in withdrawalRequest, out
        withdrawalResponse);
    newEvent = withdrawalResponse; outstandingEvent = true;
Request Query: -- 向银行服务发送查询请求
    send (promptMessageQueue, displayWait);
    send (Banking Service, in queryRequest, out queryResponse);
    newEvent = queryResponse; outstandingEvent = true;
Request Transfer: -- 向银行服务发送转账请求
    send (promptMessageQueue, displayWait);
    send (Banking Service, in transferRequest, out
        transferResponse);
    newEvent = transferResponse; outstandingEvent = true;
Dispense: -- 分发现金并更新交易状态
    ATMTransaction.updateTransactionStatus (withdrawalOK);
    send (cashDispenserInterface, in cashAmount, out dispenseStatus);
    newEvent = cashDispensed; outstandingEvent = true;
Print: -- 打印凭条并向银行服务发送确认
    send (promptMessageQueue, displayCashDispensed);
    send (Banking Service, in confirmRequest);
    send (receiptPrinterInterface, in receiptInfo, out
        printStatus);
    newEvent = receiptPrinted; outstandingEvent = true;
Eject: -- 弹出ATM卡
    send (cardReaderInterface, eject);
Confiscate: -- 没收ATM卡
    send (cardReaderMessageBuffer, confiscate);
    ATMTransaction.updatePINStatus (status);
Display Ejected: -- 显示卡被弹出的提示
    send (promptMessageQueue, displayEjected);
Display Confiscated: -- 显示卡被没收的提示
    send (promptMessageQueue, displayConfiscated);
...
end case;
end while;
end loop;

```

421

21.15.3 银行服务任务的事件顺序逻辑示例

“银行服务”从所有的 ATM 客户端接收消息（见图 21-36）。虽然它们之间的通信方式是带回复的同步通信，“银行服务”仍需建立一个消息队列，因为它需要接收来自于多个客户端的消息。在这个顺序性解决方案中，“银行服务”是一个顺序性服务任务，它处理完一个请求后才能处理下一个请求。

```

loop
receive (ATMClientMessageQ, message) from Banking Service Message Queue;
Extract message name and message parameters from message;
case Message of
Validate PIN:
    -- 验证ATM卡是否合法以及客户所输入的PIN码是否与服务器记录相匹配
    PINValidationTransactionManager.ValidatePIN
        (in CardId, in PIN, out validationResponse);
    -- 如果验证成功，那么验证响应是合法并且返回当前借记卡可以访问的账户号
    -- 否则验证响应是非法

```

422

```
    reply (ATMClient, validationResponse);
Withdrawal:
    -- 检查是否超过每日取款上限以及客户账户中是否有足够的资金来满足请求
    -- 如果所有检查都通过了, 那么将取款记入账户
    WithdrawalTransactionManager.withdraw
        (in AccountNumber, in Amount, out withdrawalResponse);
    -- 如果通过, 那么取款响应是
    -- {successful, amount, currentBalance};
    -- 否则取款响应是 {unsuccessful};
    reply (client, withdrawalResponse);
Query:
    -- 读取账户余额
    queryTransactionManager.query
        (in accountNumber, out queryresponse);
    -- 查询响应包括当前余额、最后一次存款金额 (支票账户) 或利息 (储蓄账户)
    reply (client, queryResponse);
Transfer:
    -- 检查客户的转出账户中是否有足够的资金来满足本次请求
    -- 如果通过, 那么从转出账户中扣款并加到转入账户中
    transferTransactionManager.transfer (in fromAccount#,
        in toAccount#, in amount, out transferResponse);
    -- 如果通过, 那么转账响应是
    -- {successful, amount, Current Balance of From Account};
    -- 否则转账响应是 {unsuccessful};
    reply (client, transferResponse);
Confirm:
    -- 确认取款交易成功完成
    withdrawalTransactionManager.confirm (in accountNumber, in amount);
Abort:
    -- 终止转账交易
    withdrawalTransactionManager.abort (in accountNumber, in amount);
end case;
end loop;
```